Biological Sequence Analysis

Study Notes – Working Draft

Stefan Eder

Based on the lecture *Biological Sequence Analysis* (365.060; P. Seidl, G. Klambauer, A. Regl; JKU Linz), WS 2026/27

These notes are written for people who, like the author, come to this course from computer science and not from biology. Every biological idea is explained from scratch, with analogies to things a programmer already knows, and every scoring idea comes with a small example with actual numbers. The notes combine the lecture slides with what was *said* in the recorded lectures — explanations, side remarks, and hints about what matters for the exam. All worked numbers were re-computed by script. Use the notes as a study aid alongside the slides, not as a replacement for them.

Working draft: Chapters 1–9 (lectures 1–4: biology intro, scoring, PAM, BLOSUM, tools) are written out. The remaining chapters (pairwise and multiple alignment, phylogenetics, HMMs, AI-based sequence models) are outlines and will be filled in as the lecture progresses.

If you spot an error or a gap, please open an issue or pull request at **github.com/steveTgit/AI_Master** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

October 21, 2026

Contents

1 Course Overview and Motivation . . . 1
- 1.1 What Is Sequence Analysis? . . . 1
- 1.2 Organisation of the Course . . . 2
- 1.3 Why Should We Care? Motivation . . . 2
- 1.4 Chapter Checklist . . . 4

2 Biology Basics I: Cells, the Central Dogma, DNA and RNA . . . 5
- 2.1 The Cell . . . 5
- 2.2 The Central Dogma of Molecular Biology . . . 8
- 2.3 DNA . . . 9
- 2.4 Single Nucleotide Polymorphisms . . . 12
- 2.5 RNA . . . 13
- 2.6 "Junk DNA"? Coding and Non-Coding DNA . . . 13
- 2.7 Chapter Checklist . . . 14

3 Biology Basics II: From Genes to Proteins . . . 15
- 3.1 Proteins: The Machines of the Cell . . . 15
- 3.2 Amino Acids . . . 15
- 3.3 The Peptide Bond . . . 16
- 3.4 Protein Structure . . . 17
- 3.5 Transcription: From DNA to RNA . . . 18
- 3.6 Translation: From RNA to Protein . . . 19
- 3.7 Reading Frames . . . 20
- 3.8 Protein Folding . . . 20
- 3.9 Splicing . . . 21
- 3.10 Replication: From DNA to DNA . . . 21
- 3.11 Sequencing and the Human Genome Project . . . 22
- 3.12 Mutations, Disease and Gene Expression . . . 22
- 3.13 Outlook: What This Course Is About . . . 22
- 3.14 Chapter Checklist . . . 23

4 Alignments, Similarity and Evolution . . . 24
- 4.1 What Is an Alignment? . . . 24
- 4.2 How Sequences Change: Mutations . . . 25
- 4.3 Why Align? Similarity Means Relationship . . . 25
- 4.4 Example: Hemoglobin Across Species . . . 25
- 4.5 What Does "98 % of the Genome" Mean? . . . 26
- 4.6 Sequence Is Not Everything . . . 27
- 4.7 Chapter Checklist . . . 28

5 Scoring Alignments: The Basics and DNA . . . 29
- 5.1 Which Alignment Is Better? . . . 29

5.2 Scoring Matrices . . . 30
5.3 Two Small Examples . . . 30
5.4 How Many Alignments Are There? . . . 31
5.5 Scoring for DNA . . . 32
5.6 Scoring for Proteins: The Idea . . . 34
5.7 Chapter Checklist . . . 35

6 Log-Odds Scores: Turning Counts into Scores . . . 36
6.1 Odds and Log-Odds . . . 36
6.2 The Score of a Pair of Amino Acids . . . 36
6.3 A Complete Example . . . 37
6.4 Reading a Scoring Matrix . . . 38
6.5 From a Score Back to a Probability . . . 39
6.6 Chapter Checklist . . . 40

7 PAM Matrices . . . 41
7.1 History and Name . . . 41
7.2 The Basic Idea . . . 41
7.3 From Proteins to Counts . . . 42
7.4 The Mutation Probability Matrix . . . 43
7.5 From M to PAM Scores . . . 44
7.6 Sidestep: Markov Matrices . . . 45
7.7 PAMs for Longer Time Spans . . . 45
7.8 PAM in Practice . . . 47
7.9 Problems with PAM . . . 47
7.10 Chapter Checklist . . . 48

8 BLOSUM Matrices and How to Choose a Matrix . . . 49
8.1 Why Blocks? Similarity Is Not Uniform . . . 49
8.2 The BLOSUM Idea . . . 50
8.3 Building a BLOSUM, Step by Step . . . 52
8.4 What BLOSUM62 Looks Like . . . 54
8.5 PAM versus BLOSUM, and How to Choose . . . 56
8.6 Outlook: More Ideas on Scoring and Alignment . . . 58
8.7 Chapter Checklist . . . 59

9 Tools in Practice: From Sequence to Structure . . . 60
9.1 Databases: Where Sequences and Structures Live . . . 60
9.2 BLAST . . . 61
9.3 AlphaFold . . . 62
9.4 ChimeraX . . . 63
9.5 A Real Case: METTL3 and Bladder Cancer . . . 64
9.6 Chapter Checklist . . . 65

10 Pairwise Alignment I: Global Alignment . . . 67

11 Pairwise Alignment II: Local Alignment . . . 68

12 Multiple Sequence Alignment . . . 69

13 Phylogenetics . . . 70

14 Hidden Markov Models . . . 71

15 AI-Based Biological Sequence Models 72

1

Course Overview and Motivation

Lecture: VO01 (Bio-Intro I), slides 1–22. Recording: first 20 minutes.

This course is about *biological sequences* — long strings over small alphabets that encode how living things are built — and about the algorithms we use to compare them. Before any of that makes sense we need two things: a rough idea of what the course looks like, and a reason to care. This chapter provides both.

1.1 What Is Sequence Analysis?

The slides open with a definition:

Sequence analysis

Sequence analysis is the "process of applying analytical methods to DNA, RNA, or protein sequences to retrieve meaningful information. Among the sequence analysis methods, sequence alignment and similarity analysis are widely used … The main goal of sequence alignment and similarity analysis is to show similarities and differences among biological sequences."

The field was born when DNA was discovered and people suddenly had a new kind of data — strings of the letters `A`, `C`, `G`, `T` — and needed methods to analyse it. It is now a sub-field of bioinformatics. The main focus is DNA and everything derived from it: RNA is copied from DNA, and proteins are built according to RNA (Chapter 2). But most methods in this course work on *any* kind of sequence; if you understand them for DNA, you understand them for text, time series or source code.

Intuition & Mental Model

Think of sequence analysis as *diff for biology*. When you run `diff` on two versions of a file, you want to know which lines were kept, changed, inserted or deleted. Aligning two DNA or protein sequences asks exactly the same question — except that the "versions" are the same gene in two species, separated by millions of years of evolution, and we want to know how closely related they are.

1.2 Organisation of the Course

- **People.** Philipp Seidl leads the course in WS 2026/27, together with Günter Klambauer (on sabbatical this year, so everything goes through Philipp). Earlier editions were taught by Florian Sestak and by Alois Regl, who still gives the scoring lectures and whose slides much of the course is based on.
- **Format.** A full distance-learning lecture: 13 lectures, two of them in person (Bio-Intro I and the AI-based sequence models), the rest as video releases, with a Zoom Q&A roughly every third week. All material is on Moodle; general questions go to the Moodle forum, personal ones to `seidl@ml.jku.at`.
- **Exam.** Written, pen and paper, **in person** (Linz, or the distance-learning centres in Bregenz and Vienna). First date: Monday 25.01.2027; retry exams about a month later; a third one only on request. No registration means no grade.
- **Exercise.** Separate course (UE), weekly, mostly Python; students present their solutions. The exercise may run a bit ahead of the lecture.

Table 1.1 shows the schedule and how the chapters of these notes correspond to the lectures and to the files in the course folder (`slides/`, `recordings/`, `transcripts/` share the same `VOxx_. . .` names).

Date	Topic	Lecture / files	Chapter
5.10.	Bio-Intro I	VO01 `bio-intro-1`	1, 2
12.10.	Bio-Intro II	VO02 `bio-intro-2`	3
19.10.	Scoring: PAM	VO03 `scoring-1`	4, 5, 6
9.11.	Scoring: BLOSUM	VO04 `scoring-2`	6–9
16.11.	Pairwise alignment: global	–	10 (outline)
23.11.	Pairwise alignment: local	–	11 (outline)
30.11., 7.12.	Multiple alignment	–	12 (outline)
14.12.	Phylogenetics	–	13 (outline)
21.12.	HMM basics	–	14 (outline)
11.01., 18.01.	AI-based biological sequence models	–	15 (outline)
25.01.	Exam		

Table 1.1: Course schedule (WS 2026/27) and where to find each lecture in these notes. VO04 also contains the "add-ons" (tools and a case study, Chapter 9).

1.3 Why Should We Care? Motivation

1.3.1 Sequences Are the Languages of Life

The lecture starts with the 2024 Nobel Prize in Chemistry, awarded to David Baker, Demis Hassabis and John Jumper — the latter two for **AlphaFold**. Given the sequence of amino acids of a protein, AlphaFold predicts the three-dimensional shape the protein will fold into. This was a famously hard problem: people spent entire PhDs determining the 3D structure of a single protein experimentally. Knowing the shape matters, because the shape determines what a protein does — and therefore how diseases work and how drugs could act on them. As we will see in Section 3.13, AlphaFold's success rests heavily on the topic of this course: it searches for *similar sequences* and aligns them.

Life on Earth is old: single-celled organisms (prokaryotes) appeared billions of years ago, then cells with a nucleus (eukaryotes), then fungi, plants and animals. A striking event in this history

is the *Cambrian explosion*, a geologically short period in which a huge diversity of animal species appeared; why it happened so suddenly is still not fully understood. Compared with all of this, our own existence is a tiny sliver of time. And a remarkable amount of what an organism is — how it looks, which diseases it is prone to — is encoded in its DNA.

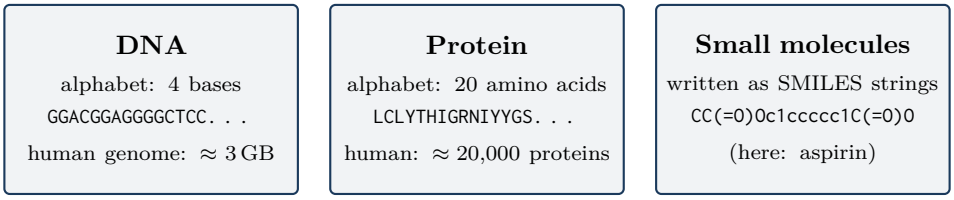


Figure 1.1: Three "languages of life" that can be written as strings: DNA, proteins and small molecules. This course deals with the first two.

If you sequenced your own genome, you would get roughly 3 GB of text over the alphabet {A, C, G, T}; compressed, about 1 GB, because much of it is repetitive. Imagine being the first person to look at that string: it is like an alien code, and we had to learn how to read it. Proteins are a second language, written in amino acids, and molecules can be written as strings too (SMILES), see Fig. 1.1.

1.3.2 Genotype and Phenotype

Genotype vs. phenotype

The **genotype** is the genetic information an organism carries in its DNA. The **phenotype** is what actually shows up: its appearance, properties and behaviour. The two are not the same — a trait can be present in the DNA without being expressed (for example a recessive gene), and the environment influences the phenotype as well.

1.3.3 Questions Sequence Analysis Can Answer

Sequence analysis gives us the tools to decode, compare and infer function from this huge amount of data; phylogenetics (Chapter 13) traces the evolutionary relationships between organisms. Typical questions:

- How do we identify genes (the parts of DNA that code for proteins) or other functional elements in a genome?
- What are the evolutionary relationships between species?
- Which mutations and genetic variants are associated with diseases?
- How do proteins evolve, and what are the functional consequences of their sequence variations?

And some concrete applications mentioned in the lecture:

- **Epidemiology.** Sequencing the DNA/RNA found in sewage water reveals which coronavirus variants circulate in a city — often before they show up in hospital data.
- **Forensics.** Companies such as 23andMe do not sequence the whole genome; they use microarrays that read only the positions that vary a lot between people. That is cheap, and it is enough to find *relatives* of a person in their databases — which has been used to solve criminal cases from DNA found at a crime scene.

- **Human origins.** DNA is chemically very stable (it can survive in bones for hundreds of thousands of years). Comparing ancient and modern DNA shows that modern humans originated in Africa and spread from there across the world.
- **Neanderthals.** Are they our ancestors? No: they are considered a separate species that split from our lineage roughly 550,000–750,000 years ago. But there was interbreeding (about 40,000–54,000 years ago), so many people today carry some Neanderthal genes.
- **The origin of SARS-CoV-2.** A study compared the virus's sequence with earlier coronavirus sequences in databases and asked how likely the observed changes are to arise naturally; it concluded that a natural origin is likely.
- And more: origins of life, evolution of diseases, and the chain *amino-acid sequence* → *3D structure* → *function* → *intervention* (drug design).

What is a species?

A common working definition: two organisms belong to the same species if they can have *fertile* offspring. *Hybrids* show that the line is blurry: a horse and a donkey can produce a mule, but the mule is (normally) sterile, so horses and donkeys are different species.

1.4 Chapter Checklist

- what sequence analysis is, and that alignment and similarity analysis are its core tools;
- course organisation: distance learning, in-person written exam, separate Python exercise;
- why the topic matters: AlphaFold, sequences as languages of life, and applications from epidemiology to forensics;
- genotype vs. phenotype; the fertile-offspring definition of a species.

2

Biology Basics I: Cells, the Central Dogma, DNA and RNA

Lecture: VO01 (Bio-Intro I), slides 23–59. Recording from about 17:00.

You do not need to become a biologist for this course — the lecturers say so explicitly. But every algorithm we will meet makes biological assumptions ("mutations are rare", "some substitutions are more likely than others", "proteins are read in triplets"), and they only make sense if you know what the strings represent. This chapter covers the first half of the biology crash course: where the information lives (the cell), how it flows (the central dogma), and how it is stored (DNA and RNA).

Intuition & Mental Model

A mental model that carries surprisingly far: a cell is a tiny **factory with a computer inside**. The **DNA** is the source code on a protected hard drive. Copies of individual "files" (genes) are made as **RNA** and sent to the factory floor, where **ribosomes** read them and assemble **proteins** — the machines that do practically all the work. Most of the biology in this course is about this pipeline.

2.1 The Cell

Cell

The cell is the smallest fundamental, self-replicating unit of life — and at the same time the largest unit that *all* life forms have in common. Every organism is either a single cell or built from many cells.

2.1.1 Prokaryotes and Eukaryotes

The first and most important distinction is a single yes/no question: does the cell have a **nucleus**, a membrane-enclosed compartment that holds the DNA?

Archaea deserve a special mention: they are prokaryotes that live in places where you would not expect life at all, for example near deep-sea hydrothermal vents at temperatures above 100 °C. Among the eukaryotes, the slides contrast three kinds of cells:

- **animal cells**: flexible boundary, no cell wall — built for mobility and consumption;

	Prokaryotes (no nucleus)	**Eukaryotes** (nucleus)
Examples	bacteria, archaea	animals, plants, fungi
DNA	floats freely in the cell	locked in the nucleus ("vault")
Size (typical)	1–5 µm (a fine dust particle)	10–100 µm (thickness of a hair)
Internal structure	few compartments	many organelles
From DNA to protein	directly: transcription and translation in the same space	extra step: the information has to be copied out of the nucleus first

Table 2.1: The two big kinds of cells.

- **plant cells**: rigid cell wall (cellulose) and chloroplasts — built for growing upwards and harvesting solar energy;
- **fungal cells**: rigid cell wall (chitin) — built to penetrate their environment and absorb nutrients.

A question from the lecture: are animal cells more closely related to plant cells or to fungal cells? Surprisingly, to **fungi**: in the tree of life, animals and fungi share a more recent common ancestor than either does with plants.

2.1.2 Inside a Eukaryotic Cell

A eukaryotic cell contains many specialised compartments, the *organelles*. You do not need to memorise their chemistry, but you should know what each one is for (Table 2.2, Fig. 2.1).

Figure to insert: labelled eukaryotic (animal) cell

A cross-section of an animal cell with the organelles from Table 2.2 labelled: nucleus, nucleolus, ribosomes, rough/smooth ER, Golgi apparatus, mitochondria, lysosomes, cytoskeleton, plasma membrane. *Search:* "eukaryotic animal cell diagram labeled organelles" (e.g. Britannica or Wikimedia Commons "Animal cell structure en.svg").

Figure 2.1: A eukaryotic cell and its organelles.

The most important point for this course is the separation of storage and production. The DNA sits in the nucleus, but the factories (ribosomes) are outside, in the cytoplasm. DNA is far too large to leave the nucleus, so the cell *copies the relevant part into RNA*, and the RNA travels out through pores in the nuclear envelope. Prokaryotes do not need this step: their DNA lies around in the same space as the ribosomes.

2.1.3 From Organelles to Organisms

Biology is organised in levels (Fig. 2.2): organelles are parts of cells, many cells form a tissue, tissues form organs, organs work together in organ systems, and all of them together make up the organism.

Cells are also extremely diverse in shape and size: a motor neuron, a red blood cell, a sperm cell and an egg cell look nothing alike. The human egg is one of the largest cells and just about

Organelle	Function	Analogy
nucleus	stores the DNA, packed into chromosomes	vault / hard drive
nucleolus	inside the nucleus; assembles ribosome subunits	
ribosome	complex of RNA and protein that builds proteins from mRNA (80S in the cytoplasm of eukaryotes, 70S in prokaryotes and mitochondria)	factory / 3D printer
mitochondria	power generators (oxidative phosphorylation); also involved in programmed cell death; have their own DNA, inherited from the mother only	power plant
chloroplasts (plants)	photosynthesis; also have their own genome	solar panels
endoplasmic reticulum (ER)	transport network; the rough ER carries ribosomes on its surface and produces proteins that are shipped onwards	road network
Golgi apparatus	processes and packages macromolecules into vesicles	post office / parcel service
lysosomes	enzymes that digest old organelles, food particles and engulfed viruses or bacteria	recycling centre
peroxisomes	get rid of toxic peroxides	waste disposal
cytoskeleton, centrosome	scaffolding; microtubules direct transport and pull chromosomes apart during cell division	scaffolding, cranes
vacuoles	storage (in animals: shipping containers; in plants and fungi: big water-filled "hydraulic cylinders")	warehouse
cytoplasm	the fluid filling the cell — crowded with proteins and other molecules	factory floor
plasma membrane	boundary that protects the inside	walls

Table 2.2: The organelles of a eukaryotic cell (after slide 27 and the lecture's explanations).



Figure 2.2: Levels of organisation, from micro to macro (e.g. mitochondrion → intestinal cell → intestinal lining → intestine → digestive system → human).

visible with the naked eye; the longest is the sciatic motor neuron, which runs from the spine to the toe (up to about 1.4 m). Mature human red blood cells are special: they have no nucleus and no DNA. And a cell looks very different depending on its stage in the cell cycle — a dividing cell, with its condensed chromosomes, looks nothing like a resting one.

2.1.4 Some Numbers

- A human consists of roughly 30×10^{12} cells (older estimates said 100 trillion).
- We carry about as many bacteria as human cells (ratio about 1.3 : 1, not the often-quoted 10 : 1), about ten times as many viruses, and a small amount of fungi (0.1 %). "We" are quite an ecosystem.
- The DNA in a single nucleus is about 2 m long when stretched out. Over all cells, that adds up to about 200 round trips from the Earth to the Sun.[1]
- Every day, 50–70 billion of our cells die in a controlled way (*apoptosis*, programmed cell death) — about 10 g of tissue.
- The first cells appeared about 3.5 billion years ago; cells were discovered by Robert Hooke in 1665.

2.2 The Central Dogma of Molecular Biology

Who tells a cell what to do, and when? Inside the cell, the work is done by **proteins**: they act as sensors, as data cables and as engines. But a cell cannot simply build everything all the time — that would waste energy. It needs strict control over *which* protein to build, *when*, and *where* to send it. That requires an organised "source code": the DNA, plus a pipeline that turns code into machines.

Central dogma of molecular biology

Genetic information flows

$$\text{DNA} \xrightarrow{\text{transcription}} \text{RNA} \xrightarrow{\text{translation}} \text{protein},$$

and DNA is copied by **replication**. The DNA is the library of instructions ("genes") for how to build proteins.

- **Replication** (DNA → DNA): the genome is copied before a cell divides (enzyme: DNA polymerase).
- **Transcription** (DNA → RNA): a gene is copied into a messenger RNA (enzyme: RNA polymerase). The "language" stays the same — nucleotides in, nucleotides out — which is why it is called a *transcription*.
- **Translation** (RNA → protein): the ribosome reads the mRNA and builds a chain of amino acids. Now the language changes — from nucleotides to amino acids — hence *translation*.

Why "dogma". Francis Crick named it so in the late 1950s. As the lecture tells it, he misunderstood the word: in science one talks of hypotheses and theories, and "dogma" sounds

[1] 3×10^{13} cells $\times$ 2 m $= 6 \times 10^{13}$ m, and one round trip is $2 \times 1.5 \times 10^{11}$ m, so about 200. Another slide says 500; the 200 is consistent with the cell count.

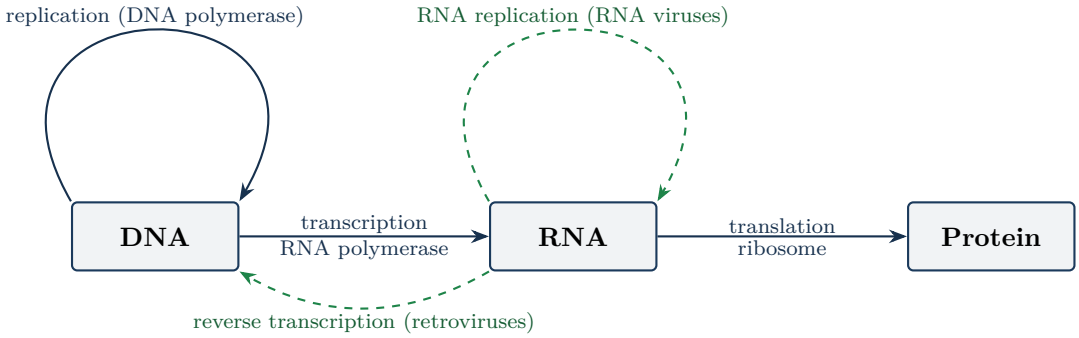


Figure 2.3: The central dogma (solid) and the most important "violations" found later (dashed): reverse transcription and RNA replication. Prions are a third, special case: misfolded proteins that pass on their *shape* to other proteins.

almost religious. The name stuck anyway — biology is full of such historical names ("chromosome" means "coloured body", because these structures took up dye under the microscope).

Exceptions. For about 20 years after the discovery of DNA, the dogma was believed to hold strictly. Since then, several exceptions have been found (Fig. 2.3):

- **reverse transcription** (RNA → DNA): retroviruses such as HIV write their RNA genome back into the DNA of the host cell;
- **RNA replication** (RNA → RNA): many RNA viruses copy their RNA directly, with an RNA-dependent RNA polymerase;
- **prions**: proteins that can change the shape of other proteins of the same kind, causing diseases such as mad cow disease (BSE) and Creutzfeldt–Jakob disease.

Prokaryotes vs. eukaryotes. In prokaryotes the pipeline is short: DNA is transcribed into mRNA, and the mRNA is translated right away. In eukaryotes there are extra steps (Chapter 3): transcription happens in the nucleus; the primary RNA transcript is processed (a protective "cap" and a poly-A "tail" are added, and the non-coding parts are *spliced* out); only then is the mature mRNA exported to the cytoplasm and translated. The extra complexity pays off: splicing allows the cell to reuse parts of a gene in different combinations (Section 3.9).

2.3 DNA

2.3.1 A Short History

The structure of DNA was identified in 1953 by James Watson and Francis Crick — with decisive help from **Rosalind Franklin**, whose X-ray image "Photo 51" revealed the helical structure. Whether Franklin got the credit she deserved is a well-known controversy. The key insight: DNA is *double-stranded*, and the two strands wind around each other into the famous **double helix**.

2.3.2 Building Blocks: Nucleotides

DNA (deoxyribonucleic acid)

DNA is a long chain of **nucleotides**. Each nucleotide consists of

- a **sugar** (deoxyribose, a ring of five carbon atoms),
- a **phosphate** group, which links the sugar to the next sugar (sugar and phosphate form the *backbone*),
- one of four **bases**: adenine (A), cytosine (C), guanine (G) and thymine (T).

The information is in the order of the bases — so for us, DNA is simply a string over the alphabet $\{\mathtt{A}, \mathtt{C}, \mathtt{G}, \mathtt{T}\}$.

Figure to insert: one nucleotide, chemically

A single nucleotide with the phosphate group, the 5-carbon sugar (carbons numbered $1'$ to $5'$) and the nitrogenous base; ideally two nucleotides of the backbone to show how phosphate links the $5'$ carbon of one sugar to the $3'$ carbon of the next. *Search:* "DNA nucleotide structure phosphate deoxyribose base 5 prime 3 prime".

Figure 2.4: The chemical structure of a nucleotide.

2.3.3 Base Pairing

The two strands are held together by **hydrogen bonds** between the bases, and only two pairings fit:

A—T (2 hydrogen bonds), G—C (3 hydrogen bonds).

So each strand completely determines the other: the strands are **complementary**. A G–C pair, with its three hydrogen bonds, is harder to break than an A–T pair. Organisms that live in harsh conditions, such as heat-loving archaea, therefore tend to have a higher **GC content**.

2.3.4 Direction: $5'$ and $3'$

The carbon atoms of the sugar are numbered $1'$ to $5'$ ("one prime" to "five prime"). In the backbone, the phosphate connects the $5'$ carbon of one sugar with the $3'$ carbon of the next. A strand therefore has a direction, from its $5'$ **end** to its $3'$ **end**, and the two strands of the double helix run in *opposite* directions (*antiparallel*, Fig. 2.5). By convention, sequences are always written from $5'$ to $3'$, and polymerases build new strands in the $5' \to 3'$ direction.

The other strand

Given the strand $5'$-ATGCGTTA-$3'$:

- Its **complement**, base by base, is TACGCAAT — but that is written $3' \to 5'$.
- Written in the conventional $5' \to 3'$ direction, the other strand is the **reverse complement**: TAACGCAT.

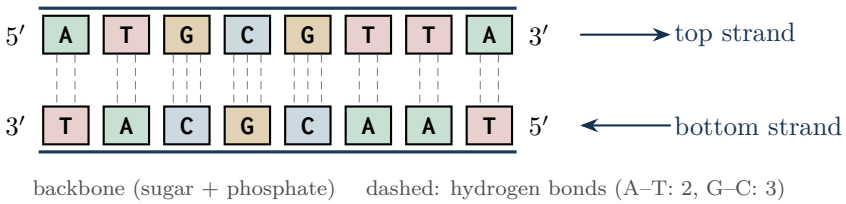


Figure 2.5: The two strands of DNA are complementary (A pairs with T, G with C) and antiparallel. The top strand reads `ATGCGTTA` from 5′ to 3′; the bottom strand, read in its own 5′ → 3′ direction, is `TAACGCAT`.

- Hydrogen bonds holding this stretch together: 5 A/T pairs ×2 + 3 G/C pairs ×3 = 19. A stretch of 8 G/C pairs would have 24, a stretch of 8 A/T pairs only 16 — that is the GC-stability argument in numbers.

Whenever a tool asks whether to search "both strands", this is the reason: a gene can sit on either strand, and on the other strand you see its reverse complement.

2.3.5 Packing Two Metres into a Nucleus

Two metres of DNA have to fit into a nucleus a few micrometres across. The cell solves this the way you would stow a long cable: it winds it up, several times over (Fig. 2.6). The DNA is wrapped around ball-shaped proteins called **histones**; the resulting "beads on a string" (chromatin) are coiled again and again, until — during cell division — they form the compact **chromosomes**.

Figure to insert: DNA packing

The levels of DNA packing: double helix → DNA wrapped around histones (nucleosomes, "beads on a string") → chromatin fibre → looped domains → condensed chromosome. *Search:* "DNA packaging histones nucleosome chromatin chromosome diagram".

Figure 2.6: How DNA is packed into chromosomes.

The histones do more than store the DNA. Their protruding "tails" carry chemical marks that decide whether a region can be unwound and read. These **epigenetic** marks let a cell switch genes on or off without changing the DNA sequence itself — changing the sequence is hard, changing how accessible it is, much easier.

Intuition & Mental Model

If the DNA is the hard drive, epigenetic marks are like *file permissions*: every cell in your body has the same files, but a liver cell and a neuron have very different files "readable". That is why cells with identical DNA can be so different (Section 3.12).

2.3.6 Chromosomes

Humans have **46 chromosomes, in 23 pairs**: one chromosome of each pair comes from the mother, one from the father. 22 pairs are the same in both sexes; the 23rd pair determines biological sex (XX: female, XY: male). Some facts mentioned in the lecture:

- The familiar X shape only appears after the DNA has been copied, just before cell division (two identical copies joined in the middle). Most of the time, chromosomes look more like sausages.
- The Y chromosome is much smaller than the X. Since men have only one X, they have no "backup copy" of the many genes on it, which is why some X-linked diseases affect men much more often.
- Mitochondria have their own small DNA (mtDNA), which is inherited from the mother only — useful for tracing maternal lineages.

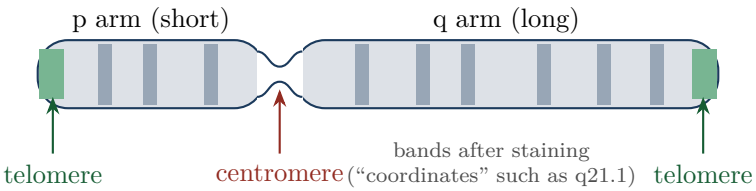


Figure 2.7: Anatomy of a chromosome. The **centromere** is the attachment point where the cell's machinery pulls the copies apart during division; it divides the chromosome into the short **p arm** and the long **q arm**. The ends are protected by **telomeres** (thousands of repeats of `TTAGGG`). The staining bands give a rough, historical coordinate system.

Telomeres and ageing. When DNA is copied, the copying machinery cannot quite reach the very end of the chromosome, so each round of replication shortens it a little. The telomeres are expendable repeats at the ends that absorb this loss. Their length therefore acts like a clock for how often a cell has divided, which is why longevity research is very interested in them. Telomeres and centromeres are mostly structural and regulatory — and, being repetitive, notoriously difficult to sequence.

2.4 Single Nucleotide Polymorphisms

No two genomes are identical (except, almost, those of identical twins), yet about 99.9 % of the DNA is the same between any two people. The differences are concentrated in specific positions.

Single Nucleotide Polymorphism (SNP)

A **SNP** ("snip") is a position in the genome where a single base varies within a population, and where the less common variant is present in more than 1 % of the population. In humans, SNPs occur roughly every 1000 bases, so each of us carries about 4–5 million of them.

SNPs are exactly what consumer DNA tests read. Some have clear effects:

- **Lactose tolerance.** Babies produce the enzyme *lactase*, which breaks down lactose (milk sugar). Keeping it switched on after weaning costs energy, so originally it was switched off. A SNP in an intron of the gene *MCM6* keeps it on — which is why most Central Europeans can digest milk as adults.

- **Diseases.** Current research associates many diseases with SNPs (e.g. schizophrenia, alcohol dependence, Crohn's disease, ALL).
- **Trade-offs.** Variants are rarely purely good or bad. Lighter skin (less melanin) makes more vitamin D but sunburns more easily; "double-muscled" cattle breeds with a disabled muscle-growth brake carry a heavy burden on their joints.

2.5 RNA

RNA is DNA's short-lived, single-stranded cousin.

	DNA	**RNA**
full name	deoxyribonucleic acid	ribonucleic acid
sugar	deoxyribose (one oxygen less)	ribose
bases	A, C, G, **T** (thymine)	A, C, G, **U** (uracil)
strands	double-stranded	single-stranded
stability	very stable (survives for ages)	unstable (degrades quickly)
length	long (millions of bases)	short (order of hundreds to thousands)
role	*stores* information	*transports* and uses it

Table 2.3: DNA versus RNA.

The instability of RNA is a feature, not a bug: a message that should only be read for a while must not last forever. (It was also a practical problem for mRNA vaccines, whose RNA had to be chemically modified to survive long enough.) There are several types of RNA:

- **messenger RNA (mRNA)**: the "classical" working copy of a gene that carries its information to the ribosome;
- **transfer RNA (tRNA)**: small adapters that bring the matching amino acid for each triplet of the mRNA (Section 3.6);
- **ribosomal RNA (rRNA)**: part of the ribosome itself, which is a mixture of RNA and proteins;
- **regulatory RNA** (e.g. miRNA, siRNA): many varieties that influence how much of which protein is made, and when.

2.6 "Junk DNA"? Coding and Non-Coding DNA

Only a small part of our genome codes for proteins. Early papers called the rest "junk DNA". That turned out to be wrong: much of it is used, just not for building proteins. Figure 2.8 shows the rough breakdown from the slides.

Intuition & Mental Model

The genome looks a lot like an old, grown codebase: only about 2 % is the actual "business logic" (the ∼20,000 protein-coding genes), surrounded by configuration and control flow (regulatory regions), modularisation (introns), and a lot of legacy code that is commented out but never deleted. One reason given in the lecture: cutting things out of the genome is risky and costly, so it is often "cheaper" to keep them.

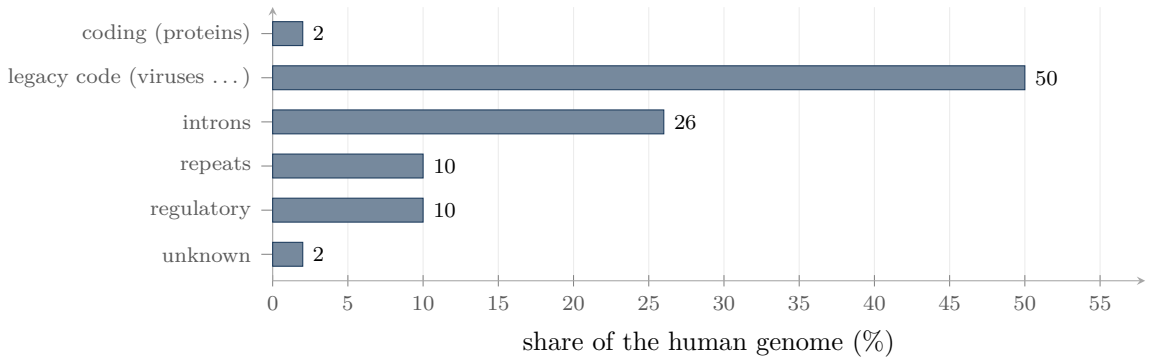


Figure 2.8: What our DNA consists of (rough estimates from the lecture). About 50 % is "legacy code" — mostly remnants of viruses, largely silenced, some repurposed for regulation; 26 % introns, which make genes modular (Section 3.9); 10 % repeats (e.g. `TTAGGG` in telomeres); 10 % regulatory elements (promoters as "start flags", enhancers as "volume dials", silencers as "brakes") and templates for non-coding RNA.

DNA is such a dense and stable storage medium that it can even be used as a hard drive: in 2017, researchers encoded a short film clip in DNA, stored it in living bacteria, and read it back by sequencing.

A note on scale and speed. Pictures of cells show lots of empty space between the organelles. In reality the cell is packed with molecules, all moving extremely fast because of thermal motion; a molecule visits practically every place in the cell within seconds. That is what makes the whole machinery work: a protein does not "find" its binding site by navigating, it bumps into it by chance, and because the bound state is energetically favourable, it stays there.

2.7 Chapter Checklist

The lecturer's tip: write the whole course onto one sheet of paper, by hand — it helps to see where each piece fits. For this chapter you should know:

- prokaryotes vs. eukaryotes (nucleus!), and what the main organelles do — especially nucleus, ribosomes and mitochondria;
- the central dogma: replication, transcription, translation, and the known exceptions;
- DNA: nucleotides (sugar, phosphate, base), the four bases, base pairing A–T (2 H-bonds) and G–C (3 H-bonds), GC content, $5' \to 3'$ direction, antiparallel strands, reverse complement;
- packing (histones, chromosomes), 23 chromosome pairs, centromere, p/q arms, telomeres, epigenetics in one sentence;
- SNPs (definition, frequency, examples);
- DNA vs. RNA, and the types of RNA;
- that only $\sim$2 % of the genome codes for proteins, and what the rest roughly is.

3

Biology Basics II: From Genes to Proteins

Lecture: VO02 (Bio-Intro II), slides 50–86 of the deck. The recording starts with a few minutes of technical problems; the lecture proper begins at about 3:45.

The first biology chapter followed the information from the nucleus to the RNA. This chapter follows it all the way to the machine: what proteins are made of, how they fold, and how the cell turns a gene into a protein (transcription and translation). Along the way we meet the concepts that the scoring chapters rely on directly: the 20 amino acids and their properties, codons, and reading frames.

3.1 Proteins: The Machines of the Cell

Protein

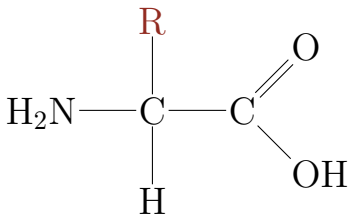
Proteins are large biomolecules made of one or more **chains of amino acids** (20 different kinds), linked by **peptide bonds**. Short chains are called *peptides* or *polypeptides*. Proteins are the molecular machines of the cell: they are involved in practically every process.

Examples from the lecture show how different their jobs are: the spike protein of SARS-CoV-2 (the target of the vaccines), the serotonin receptor (which modulates mood and digestion), **hemoglobin** (transports oxygen in the blood), cytokines (modulate immune defences), collagen (gives structure to tissues), firefly luciferase (emits light), antibodies (identify invaders) and insulin (regulates blood sugar). Others catalyse chemical reactions (enzymes such as ligases), give structure (hair), or produce movement (muscle). A small bacterium contains 50,000 to 1 million protein molecules, a human cell 1–3 billion; individual proteins live from minutes to years before they are recycled.

Example: hemoglobin. Each red blood cell contains several hundred million hemoglobin molecules. Oxygen binds to the *heme* groups in the hemoglobin, and the red blood cells carry it through the body. Hemoglobin will be our running example for sequence comparison in Chapter 4.

3.2 Amino Acids

Every amino acid has the same backbone — an *amino group* (NH_2), a central carbon atom (C_α), and a *carboxyl group* (COOH) — plus a **side chain** R that differs from one amino acid to the next (Fig. 3.1). Only the side chain changes along a protein; it gives each amino acid its chemical character (size, charge, whether it likes water).



amino group: NH_2
central carbon: C_α (with one H)
carboxyl group: COOH
side chain: R — the only part that differs

Figure 3.1: General structure of an amino acid. Glycine is the smallest one (R is just a hydrogen atom); tryptophan, with its double ring, is one of the largest.

There are 20 standard amino acids (Table 3.1), plus two rare ones (selenocysteine and pyrrolysine) that play a minor role. Learn the **one-letter codes**: protein sequences in databases and in every alignment are written with them. Several are not the first letter of the name (because that letter was already taken): F = Phe, W = Trp, Y = Tyr, K = Lys, R = Arg, D = Asp, E = Glu, N = Asn, Q = Gln. *Essential* amino acids are those the human body cannot make itself and has to get from food.

non-polar		**polar (uncharged)**		**positive charge**		**negative charge**	
glycine	Gly G	serine	Ser S	lysine	Lys K	aspartate	Asp D
alanine	Ala A	threonine	Thr T	arginine	Arg R	glutamate	Glu E
valine	Val V	tyrosine	Tyr Y	histidine	His H		
cysteine	Cys C	asparagine	Asn N				
proline	Pro P	glutamine	Gln Q				
leucine	Leu L						
isoleucine	Ile I						
methionine	Met M						
tryptophan	Trp W						
phenylalanine	Phe F						

Table 3.1: The 20 standard amino acids with three- and one-letter codes, grouped by the chemistry of their side chain as on slide 57. Non-polar side chains are *hydrophobic* (they avoid water); polar and charged ones are *hydrophilic*.

3.3 The Peptide Bond

Amino acids are linked into a chain by a **peptide bond**: the carboxyl group of one amino acid reacts with the amino group of the next, and a molecule of water (H_2O) is released (Fig. 3.2). The bond is strong (covalent).

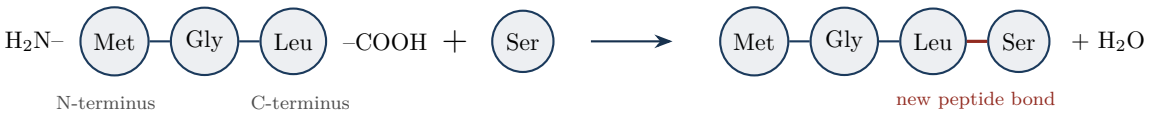


Figure 3.2: Building a protein chain. Each new amino acid is attached to the free carboxyl end (C-terminus) of the growing chain; each bond releases one water molecule.

Two consequences matter for us:

- **Proteins are linear chains.** Each amino acid has exactly one amino and one carboxyl group in its backbone, so it can connect to at most one predecessor and one successor — no

branching. That is why a protein can be written as a *string*.

- **Chains have a direction.** One end has a free amino group, the **N-terminus**; the other a free carboxyl group, the **C-terminus**. Protein sequences are written from N to C, which is also the order in which the ribosome builds them.

The reverse reaction also exists: water can attack the peptide bond, so a protein left in water long enough falls apart into its amino acids.

3.4 Protein Structure

A freshly made amino-acid chain is a floppy string. In the cell it folds into a specific three-dimensional shape, and *this shape determines what the protein does*. Biologists distinguish four levels of structure.

The four levels of protein structure

- **Primary structure**: the amino-acid *sequence* — the string.
- **Secondary structure**: local, regular folding patterns, held together by (weak) hydrogen bonds between backbone atoms: the **α-helix** (a coil), the **β-sheet** (strands lying side by side), and the irregular **loops** connecting them.
- **Tertiary structure**: how these elements are arranged in 3D to form the overall shape of *one* chain.
- **Quaternary structure**: how *several* chains (subunits) assemble into a complex. Hemoglobin, for example, consists of four chains.

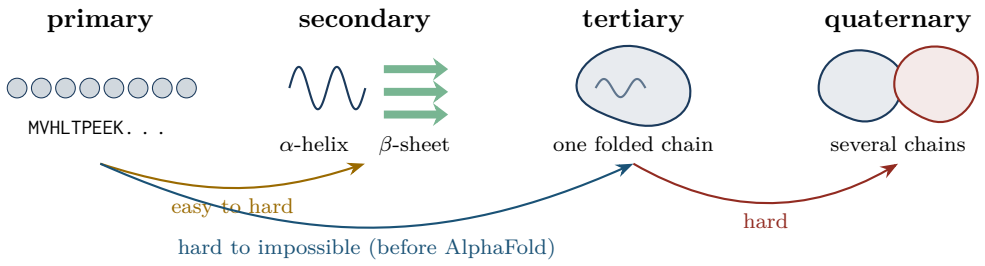


Figure 3.3: The four structure levels (schematic). The arrows show how hard it is to *predict* each level from the sequence: secondary structure is comparatively easy; the full 3D (tertiary) structure was "hard to impossible" until AlphaFold; complexes of several proteins are still hard.

Figure to insert: real protein structure levels

One protein shown at all four levels, e.g. hemoglobin: the sequence, an α-helix, one folded subunit, and the assembled four-subunit complex, in cartoon representation. *Search:* "four levels of protein structure primary secondary tertiary quaternary hemoglobin".

Figure 3.4: Protein structure levels on a real example.

Exam hint from the lecture

When showing the amino acids and the structure levels, the lecturer said: "please remember this picture ... it is also important for the exam, and that's really what you should take with you from the course." Be able to name and explain the four levels, give examples of secondary-structure elements, and explain that the side chains determine the chemistry.

Representations. A protein can be drawn as a graph of all its atoms — correct, but almost impossible to read. Bioinformaticians therefore invented the *cartoon representation*: helices as spirals or ribbons, sheets as arrows, loops as thin tubes. Experts read function from such pictures. Other views show the *solvent-accessible surface*, i.e. where a water molecule can touch the protein. None of these is "the truth"; even the atom graph is a model (electrons are really clouds of probability). We pick whichever representation helps us see what we are looking for. The same is true for sequences: a nucleotide is a molecule, but for analysing sequences the abstraction "a letter" is all we need.

Experimentally determined structures are collected in the **Protein Data Bank (PDB)**, `rcsb.org`, where each entry has a four-character code (e.g. `1A3N`, human hemoglobin). AlphaFold was trained on this data; it now predicts the coordinates of all heavy atoms directly (hydrogens are easy to add afterwards).

Prosthetic groups and ligands. Myoglobin (which stores oxygen in muscle cells) consists of several α-helices plus a *prosthetic group*: a heme group carrying the oxygen, which is *not* made of amino acids. Small molecules that bind to proteins are called **ligands**. Drug discovery is largely about finding small molecules that bind a harmful protein and switch it off — or that boost a protein the body lacks.

3.5 Transcription: From DNA to RNA

Transcription

An enzyme, **RNA polymerase**, copies one gene into a messenger RNA, in three phases:

1. **Initiation**: a complex of proteins places the RNA polymerase on the *promoter*, a start signal in front of the gene.
2. **Elongation**: the polymerase moves along the DNA, unwinds it, builds the RNA copy, and winds the DNA up again behind itself.
3. **Termination**: at a stop signal (a stop sequence and/or a hairpin loop in the RNA) the polymerase stops and releases the mRNA.

Only one of the two DNA strands is read. The polymerase reads the **template strand** and builds the RNA complementary to it. The RNA therefore has the same sequence as the other strand, the **coding strand** — with `U` instead of `T`. The RNA is built in the $5' \to 3'$ direction. The real process is far more complex (dozens of helper proteins are involved in initiation alone), but for sequence analysis this level of detail is enough.

3.6 Translation: From RNA to Protein

The ribosome reads the mRNA in steps of three nucleotides. Each triplet, called a **codon**, stands for one amino acid. The adapter between the two languages is **transfer RNA (tRNA)**: each tRNA carries one specific amino acid on one end and an *anticodon* — three bases complementary to a codon — on the other. Free tRNAs float around in the cell; only the one whose anticodon matches the current codon can bind. The ribosome then attaches its amino acid to the growing chain, shifts by one codon, and repeats.

The genetic code

- There are $4^3 = 64$ codons but only 20 amino acids, so most amino acids have several codons — the code is *redundant* ("degenerate"). Leucine, serine and arginine have six codons each; methionine and tryptophan only one.
- **Start codon:** `AUG`, which also codes for **methionine**. That is why almost every protein starts with methionine (M).
- **Stop codons:** `UAA`, `UAG`, `UGA`. They code for no amino acid; translation ends there.

1st	2nd base U	2nd base C	2nd base A	2nd base G	3rd
U	UUU Phe	UCU Ser	UAU Tyr	UGU Cys	**U**
	UUC Phe	UCC Ser	UAC Tyr	UGC Cys	**C**
	UUA Leu	UCA Ser	UAA **Stop**	UGA **Stop**	**A**
	UUG Leu	UCG Ser	UAG **Stop**	UGG Trp	**G**
C	CUU Leu	CCU Pro	CAU His	CGU Arg	**U**
	CUC Leu	CCC Pro	CAC His	CGC Arg	**C**
	CUA Leu	CCA Pro	CAA Gln	CGA Arg	**A**
	CUG Leu	CCG Pro	CAG Gln	CGG Arg	**G**
A	AUU Ile	ACU Thr	AAU Asn	AGU Ser	**U**
	AUC Ile	ACC Thr	AAC Asn	AGC Ser	**C**
	AUA Ile	ACA Thr	AAA Lys	AGA Arg	**A**
	AUG **Met**	ACG Thr	AAG Lys	AGG Arg	**G**
G	GUU Val	GCU Ala	GAU Asp	GGU Gly	**U**
	GUC Val	GCC Ala	GAC Asp	GGC Gly	**C**
	GUA Val	GCA Ala	GAA Glu	GGA Gly	**A**
	GUG Val	GCG Ala	GAG Glu	GGG Gly	**G**

Table 3.2: The standard genetic code (mRNA codons). Read the first base from the left, the second from the top, the third from the right. Start codon in green, stop codons in red. You do not need to memorise it, but you should be able to use it.

From gene to protein

Take the coding strand 5′-`ATG GCT TGG AAA GAC TAA`-3′ (spaces only for readability).

1. **Template strand** (what the polymerase reads), written 3′ → 5′: `TAC CGA ACC TTT CTG ATT`.
2. **mRNA** (complementary to the template, i.e. equal to the coding strand with U for T): 5′-`AUG GCU UGG AAA GAC UAA`-3′.

3. **Translation** with Table 3.2: AUG → Met, GCU → Ala, UGG → Trp, AAA → Lys, GAC → Asp, UAA → Stop.
4. **Protein:** Met–Ala–Trp–Lys–Asp, in one-letter code MAWKD, read from N- to C-terminus.

3.7 Reading Frames

If you are given a piece of DNA, you do not know where its codons start. Shifting the start by one or two positions gives a completely different protein; shifting by three gives the same one again. So each strand has **three reading frames**, and since a gene can be on either strand, double-stranded DNA has **six reading frames** in total.

Six reading frames of one DNA snippet

Strand: TCATGGCTTGGAAAGACTAAGC. Its reverse complement: GCTTAGTCTTTCCAAGCCATGA. Translating from position 1, 2 and 3 of each (* = stop):

frame	codons	protein
+1	TCA TGG CTT GGA AAG ACT AAG (C)	SWLGKTK
+2	CAT GGC TTG GAA AGA CTA AGC	HGLERLS
+3	ATG GCT TGG AAA GAC TAA (GC)	MAWKD*
−1	GCT TAG TCT TTC CAA GCC ATG (A)	A*SFQAM
−2	CTT AGT CTT TCC AAG CCA TGA	LSLSKP*
−3	TTA GTC TTT CCA AGC CAT (GA)	LVFPSH

Only frame +3 starts with ATG (Met) and runs until a stop codon — an *open reading frame*, the classic signature of a gene. It is the gene from the previous example, hidden in a longer piece of DNA.

Intuition & Mental Model

Reading frames are like decoding a byte stream when you have lost the byte boundaries: start one bit too late and every "byte" you read is garbage. That is why a single inserted or deleted base inside a gene (a *frameshift*) usually destroys the whole protein downstream, while a single substituted base changes at most one amino acid — and often none, thanks to the redundancy of the code.

3.8 Protein Folding

Why does a chain fold into one particular shape? Many effects play a role; the lecture highlights one of the most important ones, **hydrophobicity**. Hydrophobic ("water-fearing", non-polar) side chains tend to collapse towards the interior of the protein, where they are shielded from the surrounding water; hydrophilic (polar and charged) side chains face outward, towards the water. It is the same effect that makes oil form droplets in water and lets lipids form cell membranes.

All folding activity tends to drive the molecule into a state of lowest energy (Fig. 3.5) — folding is nature's version of an optimisation algorithm. Predicting the result of this optimisation from the sequence is exactly the problem AlphaFold solved.

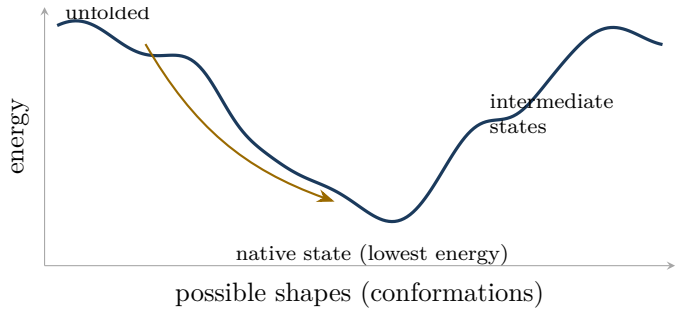


Figure 3.5: Folding as energy minimisation (schematic). The unfolded chain is high in energy; folding moves it "downhill" through intermediate states into the native state of lowest energy, in which the protein does its job.

3.9 Splicing

In eukaryotes, genes are not continuous. They consist of **exons** (the parts that end up in the protein) interrupted by **introns** (parts that are cut out). After transcription, the introns are removed from the RNA and the exons are glued together: this is **splicing**. It is done by the **spliceosome**, a machine of about 60 proteins and pieces of RNA that is assembled and disassembled for every single splicing event — and that has to work perfectly, otherwise the resulting protein is destroyed.

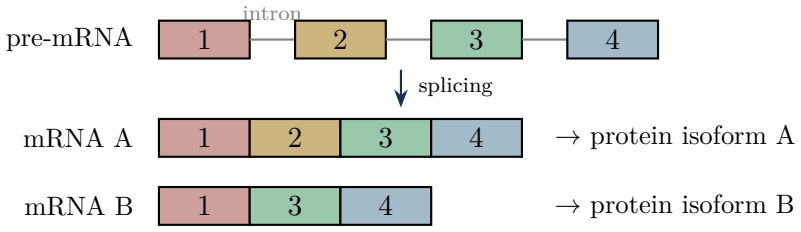


Figure 3.6: Splicing removes the introns. With **alternative splicing**, the same gene yields different mRNAs — here, one with and one without exon 2 — and therefore different proteins.

Alternative splicing

Different combinations of exons from one gene lead to different proteins (*isoforms*). This contradicts the early idea from the times of Watson and Crick of "one gene — one protein", and it is the main reason why we have many more proteins than genes. The current record holder is the *DSCAM* gene of the fruit fly *Drosophila*, which can be spliced into about 38,000 different mRNAs.

How the cell decides which exons to keep is still an active research topic; introns are thought to play regulatory roles. In software terms, introns make a gene *modular*: the exons are reusable building blocks that can be combined in different ways.

3.10 Replication: From DNA to DNA

Before a cell divides, its complete genome has to be copied, so that each daughter cell gets one copy. An enzyme called **helicase** unwinds the double helix, and **DNA polymerase** builds a new partner strand for *each* of the two old strands, using base pairing as the template. So, unlike in transcription, two strands are produced, and each new double helix consists of one old and

one new strand. Helicase is fast: the slides compare it with a jet engine, both rotating at around 10,000 revolutions per minute.

3.11 Sequencing and the Human Genome Project

DNA sequencing determines the order of the bases in a DNA molecule: cells are sampled (for example from the liver), the DNA is extracted and purified, and a sequencing machine reads it in many short pieces, which are then put together computationally (genome assembly, see Section 8.6).

The **Human Genome Project** sequenced the whole human genome. It was launched in October 1990, completed in April 2003, cost about 3 billion dollars, and was funded by the US National Institutes of Health (NIH). The result does not represent a single individual: it is a mosaic assembled from several people. Since then, the cost of sequencing has dropped dramatically — from about 100 million dollars per genome in 2001 to roughly 1,000 dollars in 2022, much faster than Moore's law would predict.

3.12 Mutations, Disease and Gene Expression

A change of a single letter can have dramatic consequences. The lecture's example is the **BRCA1** and **BRCA2** genes: more than 60 % of women who inherit a harmful change in one of them develop breast cancer during their lifetime, compared with about 13 % in the general population. For men, 7–26 % (BRCA1) and 19–61 % (BRCA2) develop prostate cancer by age 80, compared with about 10.6 % overall. Mutations can also be beneficial — lactose tolerance (Chapter 2) is an example.

Note also that every cell of your body contains the same DNA, but *not every gene is expressed in every cell*. A lung cell and a liver cell produce very different amounts of specific proteins, and that is what makes them different.

3.13 Outlook: What This Course Is About

The rest of the course is about **comparing sequences**. A typical picture is a *multiple sequence alignment*: the same protein from mouse, rat, human, pig, dog, chicken and frog written one below the other, so that corresponding positions are in the same column. Some columns are identical in all species — these positions are *conserved*, often over hundreds of millions of years, because they are essential for the protein's function. Other columns vary.

Such alignments are not just a classical tool. AlphaFold's success relies on them: for a new sequence, it first searches databases for similar sequences and builds a multiple sequence alignment, which enriches the information it feeds into its neural network. Since this database search is expensive, a current trend is to replace it with *protein language models* — neural networks trained on huge amounts of sequence data, whose internal representations are used instead. The institute behind this course contributes to this line of work, for example with Bio-xLSTM, trained on DNA and protein sequences (more in Chapter 15).

3.14 Chapter Checklist

- what proteins are and examples of what they do;
- amino-acid structure (backbone + side chain), the 20 amino acids, their one-letter codes and chemical groups (non-polar, polar, charged);
- the peptide bond, why proteins are linear chains, N- and C-terminus;
- the four levels of protein structure (α-helix, β-sheet, loops) and why structure determines function — *exam-relevant*;
- transcription (initiation, elongation, termination; template vs. coding strand) and translation (ribosome, tRNA, codons, start and stop codons, redundancy of the code); translate a short sequence by hand;
- reading frames (three per strand, six in total);
- folding by energy minimisation; hydrophobic in, hydrophilic out;
- exons, introns, (alternative) splicing;
- replication, sequencing, Human Genome Project in a few numbers.

4

Alignments, Similarity and Evolution

Lecture: VO03 (Scoring, part 1, Alois Regl), slides 1–16. Recording 0:00–27:00.

With the biology in place, we can now ask the central question of the course: *how similar are two sequences, and what does that tell us?* This chapter explains what an alignment is, why similarity between sequences is evidence for evolutionary relationship, and where that reasoning reaches its limits. The next chapters then turn "similarity" into numbers.

The scoring part of the course is given by Alois Regl, who taught the whole course for many years. His outline: first why we need alignments and a scoring system, then scoring for DNA ("the easiest of all … very primitive"), then scoring for proteins with the two classical methods PAM and BLOSUM. In practice, he says, 99.99 % of all protein alignments use one of those two. This lays the groundwork for the alignment algorithms later in the course (Chapters 10 to 12).

4.1 What Is an Alignment?

Everybody has an intuition for it: write two sequences one above the other and shift them, inserting gaps where necessary, until as many positions as possible agree.

```
THIS IS - AN AL-IGNMENT
||   ||     || |  |||||||
THAT IS NOT AN ASSIGNMENT
```

Pairwise alignment

An alignment of two sequences writes them one above the other, possibly with **gap** characters (`-`) inserted, so that both have the same length. Each column then is one of:

- a **match**: the same character in both sequences;
- a **mismatch**: two different characters;
- a **gap**: a character aligned to "nothing" — a placeholder where one sequence has no corresponding character.

A column with two gaps is never allowed.

4.2 How Sequences Change: Mutations

Alignments are interesting because sequences change over time through **mutations**. The slides illustrate the types with a word:

Mutation	Example	What happens	In an alignment
none	BEAST	—	matches
substitution	BEAST → FEAST	one letter replaced by another	mismatch
insertion	BEAST → BREAST	a letter is added	gap in the other sequence
deletion	BEAST → BEST	a letter is removed	gap in this sequence
inversion	BEAST → BEATS	a piece is reversed	not covered

Table 4.1: Types of mutations. Insertions and deletions are collectively called **indels** (they show up as gaps). Inversions — for example a whole gene that appears elsewhere in reverse order — are common in biology, but standard alignments do not model them.

4.3 Why Align? Similarity Means Relationship

The underlying idea is simple. All living things — us, animals, trees, bacteria — go back to a common ancestor (or a few). Every genome on Earth evolved from that one by small mutations that accumulated over generations. When a new species arises, it inherits the genome of its predecessor and continues to accumulate its own mutations. Mutations that are passed down the germline are the basis of evolution from the first organism to the diversity we see today.

Similarity as evidence for relationship

By comparing sequences that serve the same purpose in different organisms (typically genes or proteins), we can infer evolutionary relationships: **the more similar two sequences, the "younger" their last common ancestor**.

The rate at which mutations accumulate is called the **molecular clock**, and the simplest assumption is that it "ticks" at a constant speed. Rough estimates from the slides: in mammals about 1 mutation per 10^9 base pairs per year; mitochondrial DNA about 100 times faster; viruses 10^3 to 10^5 times faster; bacteria somewhere in between.

The clock is not constant

The constant-rate assumption is a simplification and does not really hold: there were periods with much higher mutation rates, for example the Cambrian explosion, in which many new species appeared. For our purposes it is a useful working assumption — but keep it in mind when interpreting "how long ago" from sequence differences.

Comparing sequences is not only about evolution. To find the gene responsible for a disease, one of the first steps is to align the sequences of healthy and ill people and look for the differences.

4.4 Example: Hemoglobin Across Species

Hemoglobin is an ideal example: practically all animals have it, but with slight variations accumulated during evolution. Hemoglobin from an Antarctic fish (PDB entry 1PBX) and from humans (1A3N) look almost the same at first sight, but on closer inspection there are many small differences — and their number tells us how long ago fish and humans shared an ancestor.

The lecture compares the β subunit of hemoglobin (gene *HBB*) of three organisms: human, chimpanzee (*Pan troglodytes*) and chicken. What do we expect? High similarity between human and chimpanzee (their common ancestor lived only a few million years ago — nothing compared with the billions of years of life on Earth), and lower similarity between them and the chicken. The sequences come in the standard **FASTA** format:

FASTA format

A text format for sequences. A header line starts with `>` and contains an identifier and a description; the following lines contain the sequence (for proteins in one-letter code):

```
>sp|P68871|HBB_HUMAN Hemoglobin beta OS=Homo sapiens OX=9606 GN=HBB
MVHLTPEEKSAVTALWGKVNVDEVGGEALGRLLVVYPWTQRFFESFGDLSTPDAVMGNPK
VKAHGKKVLGAFSDGLAHLDNLKGTFATLSELHCDKLHVDPENFRLLGNVLVCVLAHHFG
KEFTPPVQAAYQKVVAGVANALAHKYH
```

Human, chimpanzee and chicken hemoglobin

All three β chains have 147 amino acids. Comparing them position by position (no gaps are needed here):

- human vs. chimpanzee: **identical**, all 147 positions agree;
- human vs. chicken: 102 of 147 positions agree (69 %), 45 differ.

The first 30 positions of human and chicken:

```
MVHLTPEEKSAVTALWGKVNVDEVGGEALG
||| | |||   | |||||| | | |||
MVHWTAEEKQLITGLWGKVNVAECGAEALA
```

The multiple alignment tool *Clustal Omega* (Chapter 12) turns such comparisons into a tree. It puts human and chimpanzee at distance 0 from each other and gives the chicken a branch of length 0.153 — half of the 30.6 % difference, because in a tree the distance between two leaves is split along the path between them.

Why do all three start with M. The lecturer asked this in the lecture, and nobody knew. The start codon `AUG` codes for methionine (Section 3.6), so every protein is *made* starting with M. Many proteins lose this first methionine afterwards (*post-translational modification*), but these three still have it.

4.5 What Does "98 % of the Genome" Mean?

Headlines like "Just 2.5 % of DNA turns mice into men" (New Scientist, 2019) suggest that you could align the DNA of mice and humans and find only 2.5 % of the letters different. That is *not* what such numbers mean. Even seemingly unrelated organisms show striking similarity, simply because cells and their proteins have to work in the same or a similar way in all of them — but the percentages are defined differently.

Shared genome (the most widely accepted definition)

"A and B share x % of their genome" means: x % of the **genes** of A have a **homolog** in B. A homolog is a gene that is highly similar (in this context typically 80–90 % similar, not identical) because it derives from the same ancestral gene.

Three things follow:

1. it is about **genes only** — the non-coding regions (98 % of the DNA, Fig. 2.8) may differ much more;
2. the homologs themselves are far from identical;
3. the definition is **not symmetric** (Fig. 4.1).

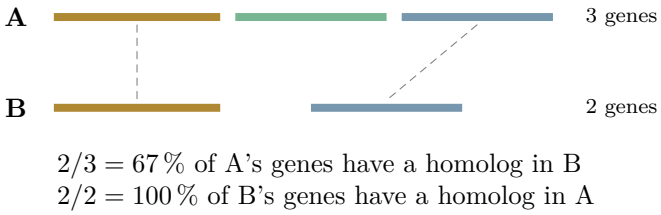


Figure 4.1: Why "shared genome" is not symmetric. Organism A has three genes, B has two; same colour = homologs. The percentage depends on whose genes you count — so always say what the 100 % are.

4.6 Sequence Is Not Everything

Are the proteins PCNA (in eukaryotes) and the β subunit of DNA polymerase III (in bacteria) related? Both have the same job: they form a ring that clamps around the DNA and keeps the DNA polymerase working while it copies. Judged by sequence, the answer is a clear "no": after alignment they share only about 10 % identical positions — about what two random sequences also reach. But their 3D structures (PDB `1AXC` and `1MMI`) look almost the same: two different ways of building the same six-domain ring (the bacterial clamp from two subunits with three domains each, PCNA from three subunits with two domains each).

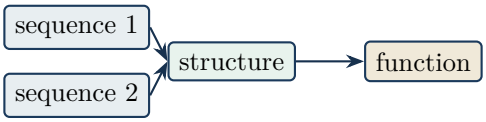


Figure 4.2: Proteins do their work based on *structure*, not sequence. The sequence determines the structure, but very different sequences can fold into the same structure — and then have the same function.

Figure to insert: PCNA and the bacterial sliding clamp

Side by side: the E. coli DNA polymerase III β clamp (PDB 1MMI) and human/yeast PCNA (PDB 1AXC), both seen from the top as a ring around the DNA — two unrelated sequences with the same shape. *Search:* "PCNA beta clamp structure comparison" or the RCSB PDB pages of 1AXC and 1MMI.

Figure 4.3: Same shape and job, almost no sequence similarity.

Be careful with sequence similarity

Sequence similarity is only part of the story. When you study evolution or function, do not rely on sequence alignment alone — look at the 3D structure as well. Today that is easy, thanks to millions of predicted structures (AlphaFold). How to *quantify* the similarity of two structures is shown in Section 9.4.

So there are two kinds of similarity: **sequence similarity** (compare the strings) and **structural (3D) similarity**. This course covers the first; structural similarity is the topic of structural bioinformatics.

4.7 Chapter Checklist

- alignment terminology: match, mismatch, gap; mutation types (substitution, insertion, deletion = indel; inversion);
- the logic "more similar ⇒ more recent common ancestor", the molecular clock, and why its constant rate is only an assumption;
- the FASTA format; why proteins start with methionine;
- what "A and B share x % of their genome" means (homologs; genes only; not symmetric);
- sequence → structure → function, and the PCNA example.

5

Scoring Alignments: The Basics and DNA

Lecture: VO03 (Scoring, part 1), slides 15–33. Recording 25:00–71:00.

To find the "best" alignment, we must first say what "best" means. This chapter introduces scores for matches, mismatches and gaps, shows how they add up to the score of an alignment, and why trying all alignments is hopeless. It then looks at how DNA is scored in practice — with a critical eye — and prepares the ground for the much more careful protein scores of the next chapters.

5.1 Which Alignment Is Better?

Here are two ways to align `ATGGCCAGC` and `AGGCCAGCA`:

```
ATGGCCAGC
| | |
AGGCCAGCA
```

without gaps: 3 matches, 6 mismatches

```
ATGGCCAGC-
| | |||||
AGG-CCAGCA
```

with gaps: 7 matches, 1 mismatch, 2 gaps

Which one is better?[1] The right one has many more matches — and matches are good, because they mean the sequence has not changed. But it needs gaps, and mutations that insert or delete letters are *rare* events compared with substitutions. Is one gap worth more matches and fewer mismatches? We cannot decide without defining how much each event is worth.

The idea of scoring

Give each column of an alignment a score: a **reward** for a match, a **penalty** for a mismatch, and a (usually larger) penalty for a gap. The score of the alignment is the sum over all columns, and the best alignment is the one with the highest total score. The scores should **imitate nature**: mutations that happen often in nature get a high (good) score, mutations that happen rarely a low (bad) one.

The answer depends on the scores

With match $= +1$, mismatch $= -2$ and gap $= -3$:

- left: $3 \cdot (+1) + 6 \cdot (-2) = -9$;
- right: $7 \cdot (+1) + 1 \cdot (-2) + 2 \cdot (-3) = -1$ $\Rightarrow$ the alignment with gaps wins.

[1] The slide counts 2 matches and 7 mismatches for the left alignment; position 5 (`C`/`C`) is also a match.

Make gaps much more expensive, say -10, and the right alignment drops to $7-2-20 = -15$, worse than the left one. The scores encode our beliefs about biology, and they decide which alignment is "optimal". (With the first set of scores, the best possible alignment of these two sequences scores $+2$: `ATGGCCAGC-` over `A-GGCCAGCA`, with 8 matches and 2 gaps.)

Reward or penalty?. Some authors speak of rewards, others of penalties. They mean the same thing with opposite signs: a penalty of 3 is a reward of -3. In practice, matches usually get positive scores and mismatches and gaps negative ones, which is where the word "gap penalty" comes from. Just be careful with the sign.

5.2 Scoring Matrices

Three numbers (match, mismatch, gap) are not enough in general: how likely a substitution is depends on *which* letters are involved. So we use a **scoring matrix** S that defines the score $S(a, b)$ for every pair of letters a, b (bases or amino acids).

The simplest approach ignores which letters are involved:

$$S(a,b) = \begin{cases} x & \text{if } a = b, \\ y & \text{if } a \neq b, \end{cases}$$

with x typically positive and y zero or negative. The very simplest idea just counts matches ($x = 1$, $y = 0$); a harder penalty for mismatches would be, for example, $x = 1$, $y = -3$. For DNA, such heuristics are used in practice; for proteins there are much more clever schemes (PAM and BLOSUM).

5.2.1 From Columns to Alignments to the Optimum

The scores are combined in three levels:

1. a **single column** gets the score $S(a_i, b_i)$ of the two letters involved (or a gap penalty);
2. a **whole alignment** gets the sum of its column scores, $\sum_i S(a_i, b_i)$;
3. the **optimal alignment** is the one with the best score among all possible alignments k:

$$\max_k \sum_i S(a_{i,k},\, b_{i,k}).$$

The last line is "what we are looking for". Finding it efficiently is the job of the alignment algorithms in Chapters 10 and 11.

5.3 Two Small Examples

5.3.1 Shifting Without Gaps

Are `ATCGA` and `GTTCG` similar? Without gaps, all we can do is shift one sequence along the other and score each position. The lecture uses match $= +1$, mismatch $= -2$, and — implicitly — charges -1 for every letter that sticks out without a partner.

All shifts of `GTTCG` against `ATCGA`

alignment	overlap	score of the overlap	total (overhang -1 each)
`ATCGA` / `GTTCG`	1	-2	-10
`ATCGA` / `GTTCG`	2	-4	-10
`ATCGA` / `GTTCG`	3	-6	-10
`ATCGA` / `GTTCG`	4	$+1$	$\mathbf{-1}$
`ATCGA` / `GTTCG`	5	-7	-7
`ATCGA` / `GTTCG`	4	-8	-10
`ATCGA` / `GTTCG`	3	-6	-10
`ATCGA` / `GTTCG`	2	-1	-7
`ATCGA` / `GTTCG`	1	-2	-10

The best shift aligns `TCG` with `TCG`:

```
ATCGA
 |||
GTTCG
```

Overlap: $-2+1+1+1=+1$; minus 1 for each of the two overhanging letters gives -1. Without gaps, there are only $5+5-1=9$ shifts to try — easy.

5.3.2 Allowing Gaps

Now compare `ATCGA` and `ACGA`, and allow insertions and deletions in both sequences, with match $=+1$, mismatch $=-2$, gap $=-3$:

Some alignments of `ATCGA` and `ACGA`

alignment	columns	score
`ATCGA` / `A-CGA`	4 matches, 1 gap	$4-3=\mathbf{+1}$
`ATCGA` / `-ACGA`	3 matches, 1 mismatch, 1 gap	$3-2-3=-2$
`ATCGA` / `AC-GA`	3 matches, 1 mismatch, 1 gap	-2
`ATCGA` / `ACG-A`	2 matches, 2 mismatches, 1 gap	$2-4-3=-5$
`ATCGA` / `ACGA-`	1 match, 3 mismatches, 1 gap	$1-6-3=-8$

The best of these is `ATCGA` over `A-CGA`: delete the `T`, and everything else matches. A dynamic-programming search over *all* alignments confirms that $+1$ is the optimum.[a]

[a] The slide lists -7 for `ACGA-`; with these scores it is -8.

5.4 How Many Alignments Are There?

With gaps allowed anywhere in both sequences, the number of possible alignments explodes. For two sequences of length n, the lecture counts

$$\binom{2n}{n} = \frac{(2n)!}{(n!)^2} \approx \frac{2^{2n}}{\sqrt{\pi n}}$$

alignments (the approximation follows from Stirling's formula).

For $n=5$ you could write a program that tries all 252 alignments. For two proteins of length 1000 — a perfectly ordinary size — there are about 10^{600}; even checking 10^{582} alignments per

n	$\binom{2n}{n}$ (exact)	approximation $2^{2n}/\sqrt{\pi n}$
5	252	258
10	184,756	186,968
20	1.38×10^{11}	1.39×10^{11}
100	9.05×10^{58}	9.07×10^{58}
1000	2.05×10^{600}	2.05×10^{600}

Table 5.1: The number of alignments grows exponentially. For comparison: the whole universe contains about 10^{80} atoms.

second would take 10^{18} seconds, about twice the age of the universe. And sequences can be thousands or millions of letters long.

Brute force is out of the question

Computing all alignments to pick the best one is completely out of reach. We need clever algorithms — dynamic programming, which finds the optimum without enumerating all alignments. That is the topic of Chapters 10 and 11.

5.5 Scoring for DNA

For DNA, practitioners use simple schemes: three numbers for match, mismatch and gap, ignoring which bases are involved and any biological reasoning. You can see this in **BLAST** (Section 9.2), *the* tool for sequence similarity searches: under "scoring parameters" you choose a match/mismatch pair such as $1, -2$ and the gap costs. Written as a matrix, match/mismatch $= 1/-2$ looks like this:

	A	C	G	T
A	1	−2	−2	−2
C	−2	1	−2	−2
G	−2	−2	1	−2
T	−2	−2	−2	1

Alois Regl calls this "in my opinion, an unforgivable oversimplification" — the numbers come "out of the gut", without statistical or scientific reasoning, and it would be interesting to see how the zillions of past alignments would change with a better scheme.

5.5.1 Transitions and Transversions

A first step towards a better world comes from chemistry. The four bases come in two shapes:

- **purines**: `A` and `G` — larger, two-ring structure;
- **pyrimidines**: `C` and `T` — smaller, single ring.

Transition and transversion

A **transition** is a substitution within the same class (purine → purine, i.e. `A` ↔ `G`, or pyrimidine → pyrimidine, `C` ↔ `T`). A **transversion** changes the class (purine ↔ pyrimidine).

Structurally, replacing a base by one of the same shape disturbs the DNA much less: `A` is a better replacement for `G` than for `C` or `T`. And the data agree (Fig. 5.1).

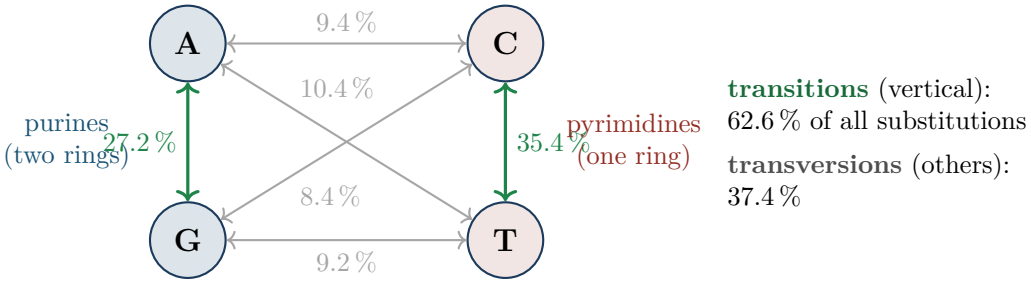


Figure 5.1: Observed substitutions between the four bases in genes of the malaria mosquito *Anopheles* (data from the slides, Wondji et al. 2007). There are two kinds of transitions but four kinds of transversions; per kind, transitions are about $31.3/9.35 \approx 3.3$ times as frequent.

So transitions are overrepresented by a factor of about 3, and transversions should get a lower score (a higher penalty). Some DNA matrices take this into account; others, including the standard BLAST settings, do not:

	A	C	G	T
A	1	−5	−1	−5
C	−5	1	−5	−1
G	−1	−5	1	−5
T	−5	−1	−5	1

transitions −1, transversions −5

	A	C	G	T
A	1	−3	−3	−3
C	−3	1	−3	−3
G	−3	−3	1	−3
T	−3	−3	−3	1

	A	C	G	T
A	2	−3	−3	−3
C	−3	2	−3	−3
G	−3	−3	2	−3
T	−3	−3	−3	2

both used by BLAST (no transition/transversion distinction)

Even the left matrix only reflects a "gut feeling" about how much worse a transversion is; there is no underlying theory behind its entries. (The BLAST matrices do have some theoretical background, explained with protein scoring in the next chapters — interestingly, they ignore transitions/transversions.)

5.5.2 NCBI's Rule of Thumb

The one guideline that exists comes from NCBI[2]: **the more similar your sequences are, the higher the ratio between the match and the mismatch score should be.**

expected similarity	99 %	95 %	75 %
match/mismatch	1/−3	1/−2	1/−1
ratio	0.33	0.50	1.00

Intuition & Mental Model

Why harsher mismatch penalties for *similar* sequences? If two sequences are expected to be 99 % identical (say, human and chimpanzee), mismatches are rare and surprising — an alignment that needs many of them is probably the wrong alignment, so it should be punished. For distantly related sequences (75 %), mismatches are everyday events and should not hurt much. The same idea returns for proteins as the choice between PAM and BLOSUM matrices for different evolutionary distances (Section 8.5).

[2]The *National Center for Biotechnology Information*, part of the US National Institutes of Health; it runs BLAST and many central databases. Its European counterpart is the EMBL (European Molecular Biology Laboratory) with its bioinformatics institute EMBL-EBI.

A question from the lecture: are these matrices always symmetric. In practice yes: $S(a, b) = S(b, a)$. In principle one could penalise `A` → `C` differently from `C` → `A`, but when we look at two sequences we cannot tell which one was the original (Section 7.1).

5.5.3 What About RNA?

Why does nobody align RNA sequences in the same way? Because RNA is normally just a copy of DNA: aligning the RNA gives the same result as aligning the corresponding DNA. What is interesting about RNA is its *structure*: most RNA types (not mRNA) fold back onto themselves, because some stretches are complementary to other stretches of the same molecule and pair up into hairpins and loops. These structures influence transcription and translation, so in RNA analysis one looks for regions of *self-complementarity*.

5.6 Scoring for Proteins: The Idea

For proteins, a two-number scheme would be far too crude. The 20 amino acids differ a lot: glycine is the smallest (its side chain is a single hydrogen atom), while arginine and tryptophan are large (tryptophan is the only one with a double ring). Some substitutions are therefore easy for a protein to tolerate and others are not.

Geometric (conformational) considerations. Mutations occur less frequently if they disturb the overall geometry of the protein — the same argument as for transitions and transversions. Valine and threonine have side chains of almost the same shape: one can replace the other without moving the chain. Replacing a small amino acid (glycine) by a large one (tryptophan) means the chain has to make room for several positions to the left and right — unlikely. The opposite direction, large to small, is easier.

Chemical considerations. But structure is not everything. Valine is *non-polar* (hydrophobic) and threonine is *polar* (hydrophilic) — in Regl's words, "like God and the devil". Polar amino acids sit on the outside of the protein, where the water is; non-polar ones hide inside. So swapping valine for threonine causes no conformational stress but a *chemical* stress, and is unlikely. Changes that tend to preserve the chemical properties of the residue, on the other hand, are common and should get a low penalty:

- non-polar ↔ non-polar and polar ↔ polar;
- aspartate ↔ glutamate (both negatively charged);
- alanine ↔ valine, leucine ↔ isoleucine (similar hydrophobic side chains);
- plus a few others.

5.6.1 Who Can Build a Substitution Matrix?

For proteins we need a 20×20 matrix with a score for every pair of amino acids — 400 entries (210 different ones, since the matrix is symmetric). Two ways to get it:

- An informed **chemist or biologist** could set up a reasonable matrix by looking at the chemical and biological background, giving high penalties to changes that cause structural or chemical difficulties. But that means reasoning about 400 numbers by hand.
- As **bioinformaticians**, we can simply look at what nature did. Our databases contain millions of protein sequences; aligning related ones and counting which substitutions actually

occurred gives the scores by statistics. The basic idea: *if a mutation is rarely seen in nature, it must be unfavourable.*

Recipe for a protein scoring matrix

1. Take a **careful selection of sequences** from one or several databases (not simply "all of them": mixing bacteria, viruses and humans leads to nothing, and the same protein appearing in many databases would be overrepresented).
2. Build alignments and **count** how often each pair of amino acids is aligned (the pairwise mutation rates).
3. **Convert** the counts into a statistical score — for PAM and BLOSUM, a *log-odds* score.

The two established implementations of this recipe are **PAM** (old, from 1974, but still widely used) and **BLOSUM** (younger, better data, more sophisticated).

5.7 Chapter Checklist

- why we need scores; reward vs. penalty; scores should imitate nature;
- scoring matrix $S(a, b)$; score of an alignment = sum of column scores; optimal alignment = maximum over all alignments;
- score the small examples by hand (shifts without gaps; alignments with gaps);
- the number of alignments ($\binom{2n}{n} \approx 4^n/\sqrt{\pi n}$) and why brute force is impossible;
- DNA scoring in practice (BLAST match/mismatch); purines vs. pyrimidines; transitions vs. transversions (factor ≈ 3); NCBI's rule of thumb;
- why protein scoring needs a 20×20 matrix: geometric and chemical reasons; the three-step recipe.

6

Log-Odds Scores: Turning Counts into Scores

Lecture: End of VO03 (from 71:00) and VO04 (31:00–50:00), slides 34–39 of the scoring deck.

The recipe from the last chapter ends with "convert the counts into a statistical score". Both PAM and BLOSUM do this in the same way: with *log-odds* scores. This chapter explains what that means. Regl called these slides the most important ones of his part of the course — once you understand the score of a single pair of amino acids, PAM and BLOSUM are just two ways of collecting the data.

6.1 Odds and Log-Odds

Consider an experiment with only two possible outcomes: success or failure, heads or tails.

Odds and log-odds

For an event A with probability $p(A)$, the **odds** are the ratio of the probability of success to the probability of failure,

$$\text{odds}(A) = \frac{p(A)}{p(\neg A)} = \frac{p(A)}{1 - p(A)},$$

and the **log-odds** are simply their logarithm, $\log\big(p(A)/(1-p(A))\big)$.

Throwing a six

Let $A =$ "throwing a 6" with one die. Then $p(A) = 1/6$ and $p(\neg A) = 5/6$ (throwing 1 to 5). The odds are

$$\frac{1/6}{5/6} = \frac{1}{5} = 0.2,$$

and the log-odds are $\log_{10} 0.2 \approx -0.70$ (or $\ln 0.2 \approx -1.61$, or $\log_2 0.2 \approx -2.32$ — the base only changes the scale).

6.2 The Score of a Pair of Amino Acids

For a substitution matrix we want, for every pair of amino acids (a, b), a number that says: is it *more or less likely than chance* to see a and b aligned with each other in related proteins?

Log-odds score (the common core of PAM and BLOSUM)

$$\text{Score}(a,b) = \text{round}\left(\text{factor} \times \log \frac{\text{observed}}{\text{expected}}\right),$$

where

- **observed** is the relative frequency with which a and b are found aligned with each other in our set of related sequences;
- **expected** is the probability of seeing them aligned *by chance*: $p(a) \times p(b)$, where $p(x)$ is the relative frequency of amino acid x in the sequences.

The factor and the rounding only make the numbers easier to use; the base of the logarithm does not matter (the original papers used $\log_2$, today $\log_{10}$ is also common).

Why $p(a) \times p(b)$? If the two positions had nothing to do with each other, they would behave like two independent dice: the probability of throwing two fours is $\frac{1}{6} \times \frac{1}{6}$. Comparing what we *observe* with what independent chance would give is exactly the right question: a pair that appears more often than chance must be favoured by evolution.

Strictly speaking, not "odds"

Regl points out that observed/expected is a *likelihood ratio*, not a ratio of success to failure. Everybody calls it "log-odds" anyway, and so do we. He is also not happy with the rounding: matrix entries are small integers (mostly between about -5 and $+10$), so rounding 2.6 to 3 or 1.4 to 1 is a substantial change — but everybody does it.

PAM and BLOSUM differ only in the data. Both use this formula. They differ in *which* sequences they count:

- **PAM** counts mutations along the family trees of closely related proteins (Chapter 7);
- **BLOSUM** counts substitutions in short, highly conserved blocks without gaps, independent of family trees (Chapter 8).

6.3 A Complete Example

This is the example Regl called "for sure the most important one" of his lectures. Our "database" is a tiny multiple alignment of four sequences over a three-letter alphabet:

```
ABC
ABB
CBA
ABA
```

Step 1: expected. There are $4 \times 3 = 12$ characters: 5 A, 5 B, 2 C. So

$$p(A) = \tfrac{5}{12} \approx 0.417, \qquad p(B) = \tfrac{5}{12} \approx 0.417, \qquad p(C) = \tfrac{2}{12} \approx 0.167.$$

(The slides round these to 0.41, 0.41 and 0.17.) The expected frequency of a pair is the product, e.g. $p(A)\,p(C) \approx 0.069$ and $p(A)\,p(B) \approx 0.174$.

Step 2: observed. In each column, compare every sequence with every sequence *below* it (not above — otherwise every pair is counted twice), and count the pairs, including the "non-mutations" such as A–A. Order does not matter: A–C and C–A are the same pair.

- column 1 (A, A, C, A): AA, AC, AA, AC, AA, CA ⇒ 3 AA, 3 AC;
- column 2 (B, B, B, B): 6 BB;
- column 3 (C, B, A, A): CB, CA, CA, BA, BA, AA ⇒ 1 BC, 2 AC, 2 AB, 1 AA.

With N sequences, each column contributes $\binom{N}{2} = N(N-1)/2$ pairs — here $4 \cdot 3/2 = 6$ — so the total is $3 \times 6 = 18$ pairs.

Step 3: compare.

pair	count	observed	expected $p(a)p(b)$	obs/exp	$\log_2$(obs/exp)
AA	4	$4/18 = 0.222$	0.174	1.28	+0.36
AB	2	$2/18 = 0.111$	0.174	0.64	−0.64
AC	5	$5/18 = 0.278$	0.069	4.00	+2.00
BB	6	$6/18 = 0.333$	0.174	1.92	+0.94
BC	1	$1/18 = 0.056$	0.069	0.80	−0.32
CC	0	0	0.028	0	$-\infty$

In this "database", A and C are aligned four times as often as chance would predict: a strongly favoured substitution (score +2 bits). A and B are aligned less often than chance: an unfavourable one. And C–C never occurs, which gives $-\infty$ — a sign that 4 sequences of length 3 are far too little data.

How much data is enough

A question from the lecture: two sequences would be "crazy"; is there a minimum? Regl knows no formal rule; his gut feeling is at least 50–100, better 200 sequences — and in general, use as many as you can, the software does not care whether it processes 10 or 10,000. Real matrices also never contain $-\infty$, because they are built from far more data. Note too that BLOSUM computes the expected frequency of an unordered pair $a \neq b$ as $2\,p(a)\,p(b)$ (the pair can arise as ab or as ba), which lowers the off-diagonal scores by one bit (Section 8.3.5).

6.4 Reading a Scoring Matrix

Ignoring the factor and rounding, a score is log(observed/expected). Every logarithm, whatever its base, is zero at 1, negative below 1 and positive above 1 (Fig. 6.1). So the *sign* of a score tells you everything:

Interpreting a score

- Score$(a, b) > 0$: $a \leftrightarrow b$ is observed **more often** than by chance — an acceptable, favoured substitution;
- Score$(a, b) = 0$: observed **as often** as by chance;
- Score$(a, b) < 0$: observed **less often** than by chance — evolution resists this substitution.

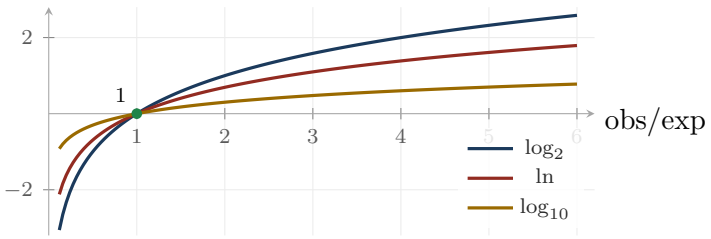


Figure 6.1: All logarithms are 0 at obs/exp = 1: the sign of a log-odds score does not depend on the base.

Reading real scores

A few entries of a real matrix (BLOSUM62):

	A	C	D	E
A	4	0	−2	−1
C	0	9	−3	−4
D	−2	−3	6	2

- A → D: −2, less likely than by chance;
- D → E: +2, more likely than by chance (both are negatively charged — a conservative substitution);
- A → C: 0, as likely as by chance;
- the diagonal is positive (an amino acid "mutating" into itself is the most common event), and C–C is particularly high: cysteine is rare and rarely replaced.

What time span does a matrix represent. A student asked whether "D → E is more likely than by chance" refers to some unit of time. Answer: it depends on the sequences used to build the matrix. Built from human and fish proteins, whose last common ancestor lived perhaps 200 million years ago, the matrix reflects that time span. PAM is explicitly designed for a defined time span (Chapter 7); the standard BLOSUM matrices were built from sequences from all over the databases, so they represent a mixture of time spans.

6.5 From a Score Back to a Probability

The formula can be inverted. Ignoring factor and rounding, with $\log_{10}$:

$$\text{Score} = \log_{10} \frac{\text{observed}}{p_a\, p_b} \quad \Longleftrightarrow \quad \text{observed} = 10^{\text{Score}} \times p_a \times p_b.$$

Score 3

With Score = 3 and $p_a = p_b = 0.02$: observed $= 10^3 \times 0.02 \times 0.02 = 0.4$. (This is the slide's illustration without scaling. In a real matrix the factor matters: BLOSUM uses $2\log_2$, so there a score of 3 means obs/exp $= 2^{3/2} \approx 2.8$, not 10^3.)

6.6 Chapter Checklist

- odds and log-odds; the dice example;
- the score formula round(factor × log(observed/expected)), what "observed" and "expected" mean, and why expected = $p(a)\,p(b)$;
- compute observed and expected frequencies and the scores for a small multiple alignment by hand (Section 6.3) — *the most important slide of this part*;
- read a matrix: positive, zero, negative scores;
- invert a score into an observed frequency.

7

PAM Matrices

Lecture: VO04 (Scoring, part 2), slides 40–63 of the deck. Recording 49:00–64:00. In the lecture, Regl skipped most of the derivation slides ("you should believe me"); this chapter goes through them step by step.

PAM was the first substitution matrix for proteins built from data, and it is still in use half a century later. The idea: look at closely related proteins, count which amino acids were replaced by which during evolution, normalise the counts to a fixed amount of evolutionary change, and turn them into log-odds scores.

7.1 History and Name

PAM goes back to **Margaret Dayhoff** (1925–1983), Georgetown University Medical Center. Her activities spanned far beyond bioinformatics — planetary atmospheres, chemical bonding, phylogeny, even neural networks and pattern recognition. She founded the *Protein Information Resource* (PIR), one of the first protein sequence databases.[1] In the 1970s (the slides say 1974) she developed the substitution matrix for proteins that is still in use today.

PAM = "Point Accepted Mutation"

The name is not even proper English; "accepted point mutation" would be more accurate, but PAM is easier to say. (For a while, "percentage of accepted mutations" was also used, which comes close.)

- **Point mutation**: one amino acid is replaced by another — no insertions or deletions, no genomic rearrangements.
- **Accepted**: accepted by evolution, i.e. the mutation stays in the lineage of descendants. Many mutations die out after a generation or two; only those passed on to all following generations count.

7.2 The Basic Idea

"Look at evolutionary events and derive the log-odds for amino-acid substitutions from them." The important twist: the mutation probabilities are scaled so that they represent a fixed *amount of evolution*, namely **one accepted mutation per 100 amino acids** (1 % change). This

[1] The slide calls it "the precursor of today's PDB". More precisely, PIR is a *sequence* database that later became part of UniProt; the PDB, the *structure* database, was founded independently in 1971.

unit is called **1 PAM**. Regl translates it, very roughly, into about one million years — a gross oversimplification, since the speed of evolution is not constant, but it gives an idea.

Dayhoff's starting point: **1572 observed mutations** in evolutionary trees of **71 families of closely related proteins** (more than 85 % sequence identity).

7.2.1 Why Closely Related Sequences?

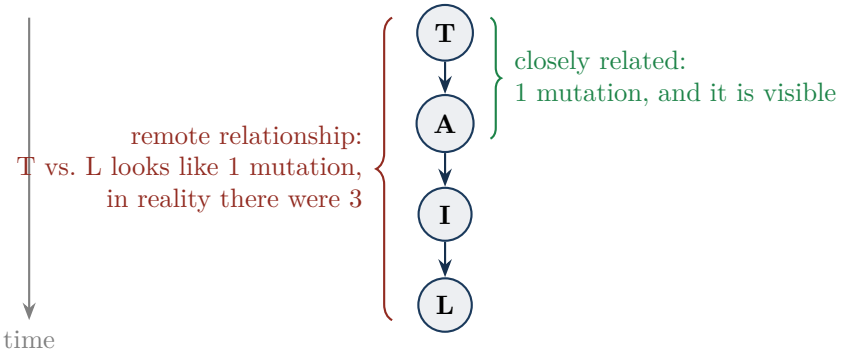


Figure 7.1: One position over time. Compared with each other, two sequences that are far apart show only the net change T → L and hide the intermediate steps. Closely related sequences make sure that each observed difference is a single event.

We want to *see* mutations, but we want to avoid *unseen* ones. The selection of sequences is a balance: not too closely related (some mutations must have happened, otherwise the counts are all zero), and not too remotely related (otherwise multiple mutations at the same position are miscounted as one, Fig. 7.1).

7.3 From Proteins to Counts

7.3.1 Counting Mutations Along a Tree

For each protein family, Dayhoff built an evolutionary tree and counted, for every branch, which amino acids were replaced by which.

Three sequences, one tree

Take the three related sequences `IAGCW`, `IAGCT` and `IIGCT`. Among all trees that could explain them, choose the one that needs the *minimum number of mutations* (Fig. 7.2): `IIGCT` differs from the other two in position 2 (A ↔ I), and `IAGCW` differs from `IAGCT` in position 5 (W ↔ T). So the count matrix A gets one entry for A–I and one for W–T; all other entries stay zero. Repeating this for all branches of all trees fills A.

7.3.2 Making the Counts Symmetric

Mutation counts are inherently *not* symmetric: A → B and B → A can (and will) occur with different frequencies. But when we look at two sequences, say `CCGTCAG` and `CCGCCAG`, we cannot tell which one was the original — was it T → C or C → T? As the best guess, the count matrix is made symmetric:

$$A_{ij} = A_{ji} = \frac{A_{ij} + A_{ji}}{2}, \qquad \text{e.g. } \frac{8+7}{2} = 7.5.$$

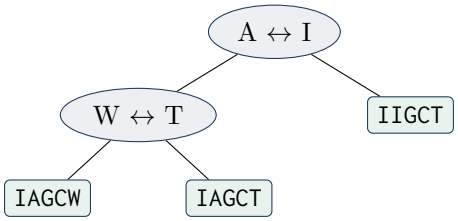


Figure 7.2: The most parsimonious tree for the three sequences: two mutations explain all differences. The arrows are two-headed on purpose — from the sequences alone we cannot tell the direction of a mutation.

7.3.3 Frequencies and Mutabilities

Quantities from the counts

- $n(j)$: total number of occurrences of amino acid j in the data; $N = \sum_j n(j)$: total number of amino acids.
- $f(j) = n(j)/N$: the **frequency** of amino acid j (sums to 1).
- $m(j) = \dfrac{\sum_{i \neq j} A(i,j)}{n(j)}$: the **mutability** of j — how often it mutated, relative to how often it occurs.

7.4 The Mutation Probability Matrix

Now we turn counts into probabilities. Let $M(a,b)$ be the probability that amino acid a is replaced by b within the time span of 1 PAM.[2]

Mutation matrix M

$$M(a,b) = \lambda \frac{A(a,b)}{n(a)} \quad (a \neq b), \qquad M(a,a) = 1 - \sum_{b \neq a} M(a,b) = 1 - \lambda\, m(a).$$

The diagonal has to be calculated, because "non-mutations" were never counted: if an amino acid does not mutate, it stays the same, so each row of M sums to 1.

Choosing λ. λ is a constant of proportionality that fixes the *amount* of evolution M represents. We want 1 mutation per 100 amino acids: on average, 99 % of the positions stay the same and 1 % change. The expected fraction of changing positions is $\sum_a f(a)\,(1 - M(a,a)) = \lambda \sum_a f(a)\, m(a)$, so

$$0.99 = 1 - \lambda \sum_{j=1}^{20} f(j)\, m(j) \quad \Longrightarrow \quad \lambda = \frac{0.01}{\sum_{j=1}^{20} f(j)\, m(j)}.$$

In matrix terms: 1 % of the probability mass sits off the diagonal and 99 % on it. Since Dayhoff's proteins were 85 % identical, $\lambda < 1$ — downweighting the observed mutations corresponds to moving towards sequences that are 99 % instead of 85 % identical.

[2] The slides write the same formulas with the roles of the indices exchanged ($M(i,j)$ as the probability that j mutates into i, so that columns sum to one). The convention used here — rows sum to one — matches the Markov-chain view in Section 7.6; the resulting scores are the same.

7.5 From M to PAM Scores

A PAM score compares the probability of an accepted point mutation in nature with the probability of seeing the same pair by chance:

- probability of seeing a replaced by b in nature: $f(a)\,M(a,b)$ (pick an a, then it mutates into b);
- probability of seeing the pair by chance: $f(a)\,f(b)$.

Their likelihood ratio is $\dfrac{f(a)\,M(a,b)}{f(a)\,f(b)} = \dfrac{M(a,b)}{f(b)}$.

The 1 PAM scoring matrix

$$S(a,b) = \log \frac{M(a,b)}{f(b)},$$

then multiplied by a factor (e.g. 10) and rounded. Because we aimed for 1 % total mutation, this matrix is called **1PAM**. It is symmetric: since $f(a)\,M(a,b) = \lambda A(a,b)/N$ and A is symmetric, $M(a,b)/f(b) = M(b,a)/f(a)$.

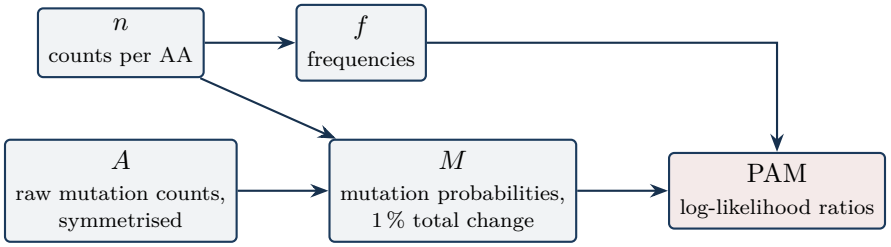


Figure 7.3: The matrices involved in PAM (the mutability m and the constant λ are hidden inside the arrow from A to M).

1PAM for a three-letter alphabet

Suppose our data contain $n = (50, 30, 20)$ occurrences of A, B and C ($N = 100$, so $f = (0.5, 0.3, 0.2)$), and the symmetrised count matrix has $A(A,B) = 4$, $A(A,C) = 2$, $A(B,C) = 1$.

1. Mutabilities: $m(A) = (4+2)/50 = 0.12$, $m(B) = (4+1)/30 \approx 0.167$, $m(C) = (2+1)/20 = 0.15$.
2. $\sum_j f(j)\,m(j) = 0.06 + 0.05 + 0.03 = 0.14$, so $\lambda = 0.01/0.14 \approx 0.0714$.
3. Mutation matrix (rows: from, columns: to):
$$M = \begin{pmatrix} 0.99143 & 0.00571 & 0.00286 \\ 0.00952 & 0.98810 & 0.00238 \\ 0.00714 & 0.00357 & 0.98929 \end{pmatrix},$$
e.g. $M(A,B) = 0.0714 \cdot 4/50 = 0.00571$ and $M(A,A) = 1 - 0.0714 \cdot 0.12 = 0.99143$. Check: $\sum_a f(a)\,(1 - M(a,a)) = 0.01$. ✓
4. Scores $10 \log_{10}(M(a,b)/f(b))$, rounded:
$$\text{1PAM} = \begin{pmatrix} 3 & -17 & -18 \\ -17 & 5 & -19 \\ -18 & -19 & 7 \end{pmatrix}.$$

For example,

$$S(A,A) = 10\log_{10}\frac{0.99143}{0.5} = 2.97 \approx 3, \qquad S(A,B) = 10\log_{10}\frac{0.00571}{0.3} = -17.2,$$

and $S(B,A)$ gives the same value.

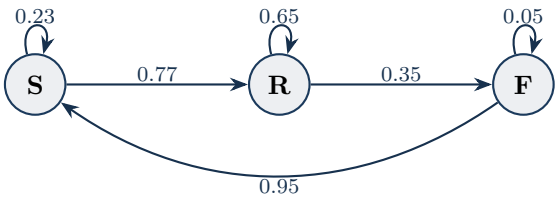
The rare amino acid C gets the highest diagonal score: seeing C aligned with C is the most "surprising" match. After only 1 % of change, all mutations are very unlikely — hence the strongly negative off-diagonal entries.

7.6 Sidestep: Markov Matrices

To get matrices for longer time spans, we need one property of PAM: it is a **Markov matrix**.

Markov (transition) matrix

Given a graph with transition probabilities along its edges, written as an $n \times n$ matrix T with T_{ij} = probability of going from node i to node j in one step: each row sums to 1 ("you have to go somewhere, even if you stay"). The probabilities for k steps are given by the k-th matrix power, T^k.



$$T = \begin{pmatrix} 0.23 & 0.77 & 0 \\ 0 & 0.65 & 0.35 \\ 0.95 & 0 & 0.05 \end{pmatrix}$$

Figure 7.4: A three-state Markov chain (the slide's example) and its transition matrix; rows and columns in the order S, R, F.

Two steps

$$T^2 = \begin{pmatrix} 0.053 & 0.678 & 0.270 \\ 0.333 & 0.423 & 0.245 \\ 0.266 & 0.732 & 0.003 \end{pmatrix}.$$

For example, $T^2_{SR} = 0.23 \cdot 0.77 + 0.77 \cdot 0.65 + 0 \cdot 0 = 0.678$: either stay in S and then go to R, or go to R and stay there. After many steps, every row converges to the same vector (T^{50} has all rows equal to $(0.249, 0.549, 0.202)$): the chain "forgets" where it started and reaches its *stationary distribution*.

7.7 PAMs for Longer Time Spans

nPAM

Since 1PAM describes one step of evolution as a Markov matrix, n steps are described by the n-th power,

$$M^{(n)} = M^n \qquad (\text{"}n\text{PAM} = 1\text{PAM}^n\text{"}),$$

and the nPAM scores are computed from M^n exactly as before,

$$S_n(a,b) = \log \frac{M^n(a,b)}{f(b)}.$$

Matrices up to 250PAM are in use. Rule of thumb for small n: with nPAM you can expect about n % mutations.

Power of the probabilities, not of the scores

"nPAM = 1PAM^n" on the slides refers to the *mutation probability matrix* M. Raising the integer *score* matrix to the n-th power would be meaningless — scores are logarithms. Compute M^n first, then take logs.

Does 100PAM mean 100 % changed? And 250PAM. No. Over long time spans, some residues change several times, some change back to their original, and some do not change at all. The higher n, the more the mutation probabilities approach 100 % — but only *asymptotically*. (In practice, 250PAM corresponds to sequences that are still about 20 % identical, Table 8.1.) The transition probabilities "walk away" from the diagonal as the time span grows (Fig. 7.5).

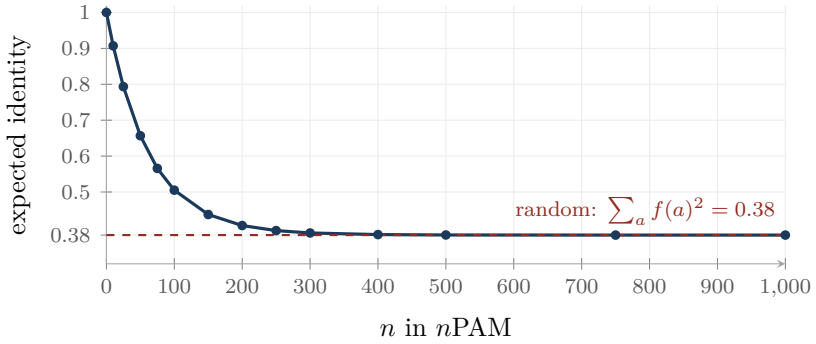


Figure 7.5: The three-letter example: probability that a position is unchanged after nPAM, $\sum_a f(a)\, M^n(a,a)$. It starts at 1, drops by about 1 % per PAM at first, and then levels off at the value two *random* sequences would have. After 100PAM, about half of the positions are still identical, not zero.

The three-letter example over time

	identity	M^n diagonal	scores $10 \log_{10}(M^n(a,b)/f(b))$
1PAM	0.99	$(0.991, 0.988, 0.989)$	diag $(3,5,7)$, off-diag $-17 \ldots -19$
10PAM	0.91	$(0.921, 0.890, 0.899)$	diag $(3,5,7)$, off-diag $-8, -9$
50PAM	0.66	$(0.711, 0.598, 0.608)$	diag $(2,3,5)$, off-diag $-2, -3$
100PAM	0.51	$(0.590, 0.428, 0.408)$	diag $(1,2,3)$, off-diag -1
250PAM	0.39	$(0.507, 0.311, 0.228)$	diag $(0,0,1)$, off-diag 0
∞PAM	0.38	$(0.5, 0.3, 0.2) = f$	all 0

The longer the time span, the less a match is worth and the less a mismatch hurts — exactly the NCBI intuition from Section 5.5.1.

Exam-style question: what does 0PAM look like? And 10,000,000PAM

Regl used to give extra points for a quick answer:

- **0PAM**: no time has passed, nothing has changed — M^0 is the **identity matrix** (1 on the diagonal, 0 elsewhere). Every amino acid is only ever aligned with itself.
- **10,000,000PAM**: after a practically infinite time the chain has forgotten where it started: every row of M^n equals the background frequencies f, so "observed = expected" for every pair and all scores become 0. The distribution is random — there is no difference between diagonal and off-diagonal any more, and an alignment carries no information.

7.8 PAM in Practice

Always use a PAM that fits your project: to compare humans with chimpanzees, use a low PAM (the slides suggest 20PAM or 50PAM; Regl also mentions 5PAM or 10PAM); to compare humans with fish, use 250PAM.

What 250PAM looks like. In the 250PAM matrix, all diagonal entries are positive. Cysteine (C, 12), tryptophan (W, 17) and tyrosine (Y, 10) have particularly high diagonal values: they strongly resist mutation. Phenylalanine and tyrosine (F–Y, +7) replace each other easily, both being large aromatic amino acids.

Minitask from the slides: find the error. The slides show a short MATLAB program that computes 1PAM for six amino acids and ask who finds the error. The relevant line is

```
p(i,j) = tsym(i,j) / 100 * frel(i) * ss;
```

where `ss` is the sum of all off-diagonal counts. From the formulas above, $\lambda = 0.01\,N/\texttt{ss}$, so the intended value is $\lambda A(i,j)/n(i) = A(i,j)/(100 \cdot f(i) \cdot \texttt{ss})$. Because of operator precedence, the code *multiplies* by `frel(i) * ss` instead of dividing — the parentheses are missing. (The diagonal also needs special treatment, $1 - \lambda m$, rather than the same formula.) A full worked PAM example is in the lecture notes ("Bioinformatics I" script, Appendix C.1.1).

7.9 Problems with PAM

- **Old data.** In 1974 only few sequences were available, so PAM is certainly biased.
- **Error amplification.** Small errors in 1PAM are blown up by repeatedly multiplying the matrix for nPAM — you cannot really trust 250PAM.
- **Missing rare mutations.** Using only closely related sequences, some rare substitutions are underrepresented or missing altogether.
- **A devil's circle.** A good PAM needs good phylogenetic trees, while good phylogenetic trees need good PAMs.
- **Position dependence.** The mutation rate is not constant along a protein; it depends on where an amino acid sits in the structure. This observation leads directly to BLOSUM.

7.10 Chapter Checklist

- what PAM stands for (accepted point mutation) and the 1PAM unit (1 mutation per 100 amino acids);
- Dayhoff's data (71 families, $> 85\,\%$ identity, 1572 mutations) and why closely related sequences are needed;
- counting along a parsimonious tree; symmetrising A; n, f, m; the mutation matrix M and the choice of λ;
- $S(a,b) = \log(M(a,b)/f(b))$; compute a small example;
- Markov matrices and nPAM $= M^n$; why 100PAM is not $100\,\%$ change; 0PAM and ∞PAM;
- choosing a PAM by evolutionary distance; the problems of PAM.

8

BLOSUM Matrices and How to Choose a Matrix

Lecture: VO04 (Scoring, part 2), slides 64–120 of the deck. Recording 64:50–78:00. As with PAM, the lecture skipped the derivation ("these are all the details how you would do that; we will skip them"); this chapter works through them with a complete small example.

BLOSUM is the second big family of substitution matrices and the one you will use in practice: it is the default in BLAST and most other tools. It uses the same log-odds idea as PAM (Chapter 6); what changes is the data it counts.

8.1 Why Blocks? Similarity Is Not Uniform

When two related proteins are compared, the similarity is usually *not* spread evenly along the sequence. In many cases it is concentrated in a few short regions — *blocks* — separated by stretches that are much less similar. The slides show this for the two DNA clamps from Chapter 4 (BLASTing 1MMI against 1AXC) and for the human TATA-box binding protein compared with other organisms: high similarity appears as isolated red blocks.

Why blocks appear. A protein can roughly be divided into its **functional site** — the handful of amino acids that actually do the work, for example the ones that bind to DNA — and the rest, which is mostly there for structural reasons, to hold everything in place. A mutation in the structural part often has little or no effect. A mutation in the functional site can be disastrous: the protein loses its function, and the organism may die. Such mutations are not "accepted" by evolution (Chapter 7). So functional sites **mutate much more slowly**, and they stay similar across species (Fig. 8.1).

The software analogy

Think of a widely used library. Its public API — function names and signatures that thousands of callers depend on — practically never changes, because any change breaks everything. Internal helper code and comments are rewritten all the time. If you compared ten versions of the library, the API would be a "conserved block". Functional sites are the public API of a protein. Histones, the proteins DNA is wrapped around (Section 2.3.5), are an extreme case: almost their entire sequence is "API", and they are among the most highly conserved proteins known.

The slides' example is **vitronectin**, an essential part of human blood serum that is also found in the bone matrix: a mutation in one region has little or no effect on its function, a mutation in another possibly a disastrous one.

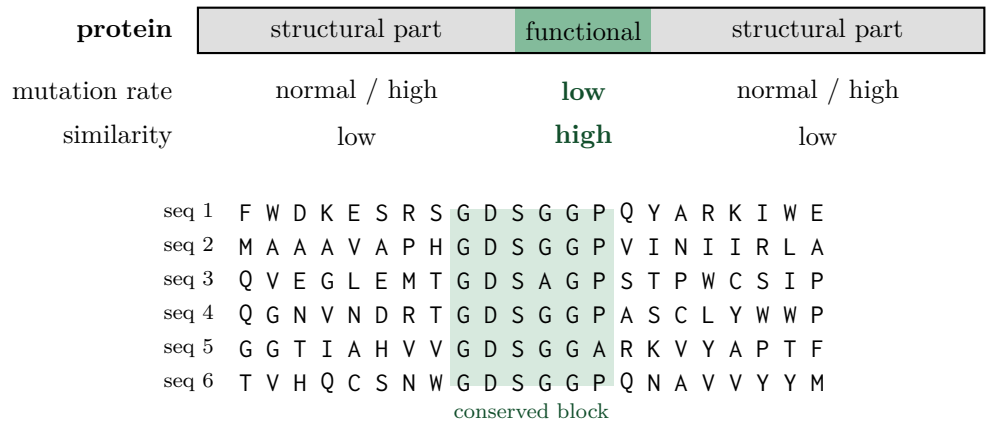


Figure 8.1: Top: the functional site of a protein mutates slowly, so it stays similar across related proteins. Bottom: in a multiple alignment of related sequences this shows up as a *conserved block* (a made-up illustration; only two positions in the block have changed).

8.2 The BLOSUM Idea

BLOSUM = BLOcks SUbstitution Matrix

A substitution matrix computed from **blocks**: short, highly conserved regions of multiple alignments of related proteins, *without gaps*. The blocks come from the BLOCKS database.[a] **BLOSUM**x is the matrix built from blocks in which sequences that are more than x % identical have been merged, so that no two remaining (groups of) sequences are more than x % similar.

[a] BLOSUM was introduced by Steven and Jorja Henikoff in 1992.

How PAMs are made	How BLOSUMs are made
Take closely related sequences (whole proteins).	Take highly conserved subsequences (blocks without gaps) from groups of proteins.
Build family trees, count mutations along the branches.	Eliminate or merge sequences that are too similar (more than x %).
Scale to 1 % change (1PAM), extrapolate by matrix powers (nPAM).	Count substitutions directly in the columns of the blocks — one calculation per matrix, no extrapolation.
Log-odds, scale, round.	Log-odds, scale (factor 2), round.

8.2.1 Blocks

A block is a set of aligned subsequences in which the columns are highly similar (or identical). Blocks

1. all have the **same length** (they are truncated if necessary), and
2. contain **no insertions or deletions** — only substitutions.

If one sequence has a gap inside a region, it is simply left out of that block (on the slide, the beetle genera *Pseudomorpha* and *Carenum* are dropped from one block each because of their gaps). How the underlying multiple alignment is made in the first place is the topic of Chapter 12.

8.2.2 "Closely Related, but Not Too Close"

The guiding principle when selecting sequences is the same as for PAM. But BLOSUM must also solve a different problem: databases are full of near duplicates.

12,000 human insulins

Searching UniProt for "insulin" gives about 292,000 results; restricted to human insulin there are still nearly 12,000 entries — insulin bound to many different molecules, in many configurations, variants and fragments. If we counted all of them, insulin would be grossly overrepresented, and the matrix would mostly describe insulin rather than proteins in general.

So first, compute all pairwise similarities (the relative number of matches after alignment, in %) and mark every pair that is at least x % similar. Groups of such close relatives form *clusters*. Then there are three ways to handle a cluster:

- **Elimination**: keep one representative (e.g. the first) and remove all others;
- **Similarization**: replace the cluster by a single *consensus* sequence (majority vote in each column);
- **Weighting**: keep all n sequences of the cluster, but give each a weight of $1/n$.

Elimination, similarization and weighting at 75 %

Seven sequences of length 4; "75 % identity" means 3 of 4 positions agree.

Input			Elimination	Similarization	Weighting
1	`VIIL`	cluster	`VIIL`	`VIIL`	`VIIL` (50 %)
2	`VIII`	cluster	—		`VIII` (50 %)
3	`LIVV`		`LIVV`	`LIVV`	`LIVV` (100 %)
4	`LLVI`		`LLVI`	`LLVI`	`LLVI` (100 %)
5	`VLLI`	cluster	`VLLI`	`VLLV`	`VLLI` (33 %)
6	`VLLV`	cluster	—		`VLLV` (33 %)
7	`VLLV`	cluster	—		`VLLV` (33 %)

Sequences 1 and 2 differ only in the last position, and so do 5, 6 and 7. Elimination keeps the first one of each cluster. Similarization takes the majority in each column: `VLLV` wins 2:1 in the second cluster; in the first, the last position is a tie, and the slide keeps `L`. Weighting can also be read as one sequence per cluster with a "fractional" last letter: `VII`[50 % L, 50 % I] and `VLL`[33 % I, 67 % V].

The BLOSUM number runs the "wrong" way

In BLOSUMx, sequences up to x % similarity are kept as separate sequences. A **high** x (BLOSUM90) keeps very similar sequences: the matrix describes **short** evolutionary distances. A **low** x (BLOSUM45) merges everything above 45 %, so only distant relationships are counted: the matrix is for **distant** relatives. For PAM it is the opposite: a high number means a long distance (Fig. 8.4).

8.3 Building a BLOSUM, Step by Step

The procedure on the slides, for sequences from the BLOCKS database with about $x\,\%$ identity:

1. cluster the sequences and merge those with more than $x\,\%$ identity;
2. count the amino-acid pairs in each column and sum them up: matrix C;
3. compute the normalising constant T and the observed pair probabilities Q;
4. compute the probabilities p of the single amino acids and from them the expected pair probabilities E;
5. compute the log-odds scores $S = \log_2(Q/E)$, scale them and round them to the nearest integer.

Everything we are aiming for is the familiar formula $S(a,b) = \log(q_{a,b}/e_{a,b})$: observed mutation probability over the probability by chance.

8.3.1 Step 1: Counting Pairs, Giving C

Take each block, look at each column separately, and count every pair of amino acids in that column — *each sequence with each sequence below it*, including identical pairs, exactly as in Section 6.3. Add up the counts over all columns (and all blocks) to get the symmetric count matrix $C = (C_{ij})$.

Possible pairings in one column

A
A
B
A
C
A

A column with four A, one B and one C. All possible pairs:

- A–A: choose 2 of the 4 A's: $\binom{4}{2} = 6$;
- A–B: each of the 4 A's with the B: $4 \times 1 = 4$;
- A–C: $4 \times 1 = 4$;
- B–C: $1 \times 1 = 1$.

Check: $6+4+4+1 = 15 = \binom{6}{2} = \frac{6\cdot 5}{2}$, the number of pairs of 6 sequences. ✓

The example already contains the shortcut. If $n_i^{(k)}$ is the number of times amino acid i occurs in column k:

Shortcut for C

$$C_{ij} = \sum_k c_{ij}^{(k)}, \qquad c_{ij}^{(k)} = \begin{cases} \binom{n_i^{(k)}}{2} = \dfrac{n_i^{(k)}\left(n_i^{(k)}-1\right)}{2} & \text{if } i = j, \\ n_i^{(k)}\, n_j^{(k)} & \text{otherwise.} \end{cases}$$

Same letter: choose two of them. Different letters: every i with every j.

8.3.2 Step 2: Normalising, Giving Q

The total number of pairs is the normalising constant. With L columns in the block and N sequences, every column contributes $\binom{N}{2}$ pairs, so

$$T = \sum_{i \geq j} C_{ij} = L \times \frac{N(N-1)}{2}, \qquad Q_{ij} = \frac{C_{ij}}{T}.$$

Q_{ij} is the **observed** probability of the pair i–j. (Sum over $i \geq j$ only: the pair A–C is the same as C–A and must not be counted twice.)

8.3.3 Step 3: Single Amino-Acid Probabilities p

$$p_i = Q_{ii} + \sum_{j \neq i} \frac{Q_{ij}}{2}.$$

Intuition: every pair consists of two letters. In a pair i–i both letters are i; in a pair i–j only half of the letters are i. So p_i is the fraction of all letters in all pairs that are i.

8.3.4 Step 4: Expected Probabilities E

If the two amino acids in a pair were independent (two dice), we would expect

$$E_{ab} = \begin{cases} p_a^2 & \text{if } a = b, \\ p_a p_b + p_b p_a = 2\, p_a p_b & \text{otherwise.} \end{cases}$$

The factor 2 is there because an unordered pair a–b can arise in two ways (a on top and b below, or the other way round), just as "a 1 and a 2" with two dice has probability 2/36, but "two 6" only 1/36.

8.3.5 Step 5: Scores and the Final Matrix

From Q and E to BLOSUM

$$S(a,b) = \log_2 \frac{Q_{ab}}{E_{ab}}, \qquad \text{BLOSUM}(a,b) = \text{round}\big(2 \times S(a,b)\big).$$

The factor 2 is the usual one for BLOSUM (scores in half-bit units), but any factor would work. As always: BLOSUM$(a,b) > 0$ means the substitution appears more often than by chance, ≈ 0 equally often, < 0 less often.

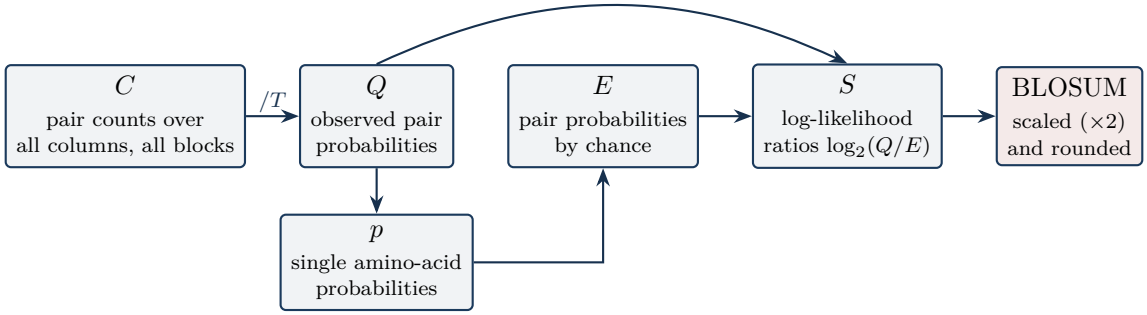


Figure 8.2: The matrices involved in BLOSUM. Compare with PAM (Fig. 7.3): only the way of collecting the data differs.

A complete BLOSUM from a tiny block

Our "block" has $L = 2$ columns and $N = 3$ sequences over the letters N, D and F:

```
NF
NF
DF
```

Step 1, counts. Column 1 has $n_N = 2$, $n_D = 1$: $c_{NN} = \binom{2}{2} = 1$, $c_{ND} = 2 \times 1 = 2$, $c_{DD} = \binom{1}{2} = 0$. Column 2 has $n_F = 3$: $c_{FF} = \binom{3}{2} = 3$. Summed: $C_{NN} = 1$, $C_{ND} = 2$, $C_{FF} = 3$, all others 0. (Check by raw counting: N–N once, N–D twice in column 1; F–F three times in column 2.)

Step 2, normalise. $T = 2 \times \frac{3 \cdot 2}{2} = 6$, so $Q_{NN} = \frac{1}{6}$, $Q_{ND} = \frac{1}{3}$, $Q_{FF} = \frac{1}{2}$.

Step 3, single probabilities. $p_N = \frac{1}{6} + \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{3}$, $p_D = 0 + \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6}$, $p_F = \frac{1}{2}$ (sum: 1 ✓; indeed 2 of the 6 letters are N, 1 is D, 3 are F).

Steps 4 and 5.

pair	Q	E	Q/E	$S = \log_2(Q/E)$	BLOSUM = round($2S$)
N–N	1/6	$p_N^2 = 1/9$	1.5	0.585	1
N–D	1/3	$2p_N p_D = 1/9$	3	1.585	3
F–F	1/2	$p_F^2 = 1/4$	2	1.000	2
D–D	0	1/36	0	$-\infty$	$-\infty$
N–F	0	$2p_N p_F = 1/3$	0	$-\infty$	$-\infty$
D–F	0	$2p_D p_F = 1/6$	0	$-\infty$	$-\infty$

N–D gets the highest score: in our data, N and D are aligned three times as often as chance predicts. The $-\infty$ entries show again that a real matrix needs vastly more data — with many blocks, the counts C are simply summed over all of them (the course script, formulas 3.22–3.24, covers several blocks; a larger worked example is in its Appendix C.1.2).

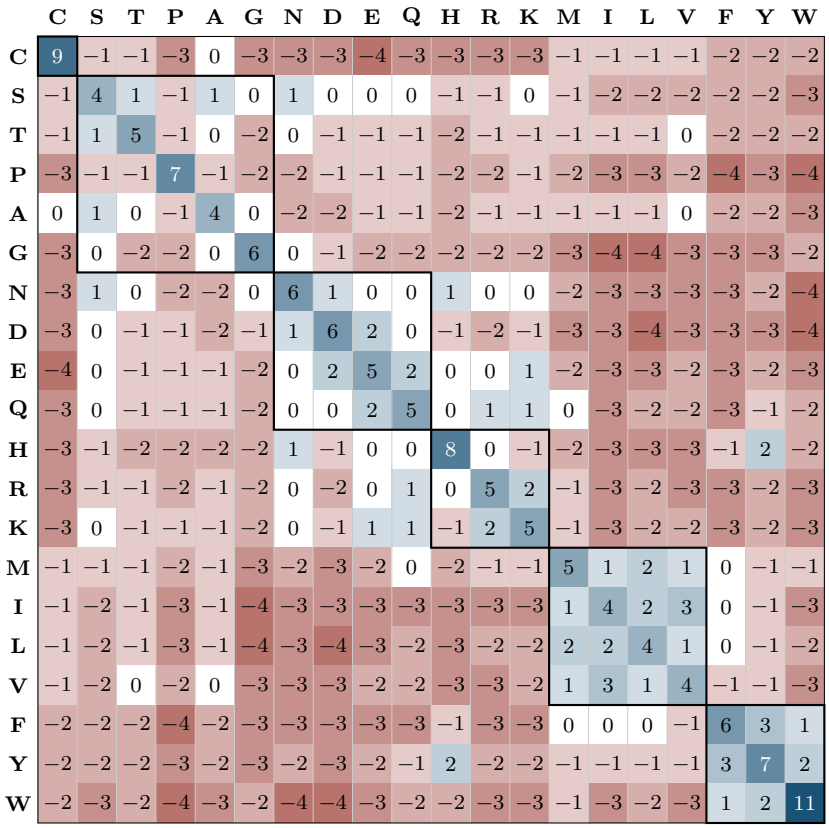
8.4 What BLOSUM62 Looks Like

Figure 8.3 shows the full BLOSUM62 matrix. A few things to read off:

- The **diagonal** is always positive. It is largest for rare, hard-to-replace amino acids: tryptophan (W, 11) and cysteine (C, 9). Common amino acids with many acceptable substitutes have small diagonal values (A, S: 4).
- Positive off-diagonal entries cluster in the **group squares**: I–L–V–M, F–Y–W, D–E–N–Q, K–R. These are the conservative substitutions from Chapter 5: similar size and chemistry.
- Replacing a hydrophobic by a charged amino acid is strongly penalised (the red areas between the groups).

Conformational aspects. The slides check some entries against chemistry:

- leucine → isoleucine: +2 — both large hydrophobic, almost the same shape;
- aspartate → glutamate: +2 — both negatively charged, glutamate is just one CH_2 group longer;
- tyrosine → phenylalanine: +3 — both aromatic rings; tyrosine has only an extra OH group;
- alanine → valine: 0 — both small and hydrophobic, yet only "as often as chance". The matrix is learned from data, not from chemistry textbooks, and it does not always match our intuition.



	C	S	T	P	A	G	N	D	E	Q	H	R	K	M	I	L	V	F	Y	W
C	9	−1	−1	−3	0	−3	−3	−3	−4	−3	−3	−3	−3	−1	−1	−1	−1	−2	−2	−2
S	−1	4	1	−1	1	0	1	0	0	0	−1	−1	0	−1	−2	−2	−2	−2	−2	−3
T	−1	1	5	−1	0	−2	0	−1	−1	−1	−2	−1	−1	−1	−1	−1	0	−2	−2	−2
P	−3	−1	−1	7	−1	−2	−2	−1	−1	−1	−2	−2	−1	−2	−3	−3	−2	−4	−3	−4
A	0	1	0	−1	4	0	−2	−2	−1	−1	−2	−1	−1	−1	−1	−1	0	−2	−2	−3
G	−3	0	−2	−2	0	6	0	−1	−2	−2	−2	−2	−2	−3	−4	−4	−3	−3	−3	−2
N	−3	1	0	−2	−2	0	6	1	0	0	1	0	0	−2	−3	−3	−3	−3	−2	−4
D	−3	0	−1	−1	−2	−1	1	6	2	0	−1	−2	−1	−3	−3	−4	−3	−3	−3	−4
E	−4	0	−1	−1	−1	−2	0	2	5	2	0	0	1	−2	−3	−3	−2	−3	−2	−3
Q	−3	0	−1	−1	−1	−2	0	0	2	5	0	1	1	0	−3	−2	−2	−3	−1	−2
H	−3	−1	−2	−2	−2	−2	1	−1	0	0	8	0	−1	−2	−3	−3	−3	−1	2	−2
R	−3	−1	−1	−2	−1	−2	0	−2	0	1	0	5	2	−1	−3	−2	−3	−3	−2	−3
K	−3	0	−1	−1	−1	−2	0	−1	1	1	−1	2	5	−1	−3	−2	−2	−3	−2	−3
M	−1	−1	−1	−2	−1	−3	−2	−3	−2	0	−2	−1	−1	5	1	2	1	0	−1	−1
I	−1	−2	−1	−3	−1	−4	−3	−3	−3	−3	−3	−3	−3	1	4	2	3	0	−1	−3
L	−1	−2	−1	−3	−1	−4	−3	−4	−3	−2	−3	−2	−2	2	2	4	1	0	−1	−2
V	−1	−2	0	−2	0	−3	−3	−3	−2	−2	−3	−3	−2	1	3	1	4	−1	−1	−3
F	−2	−2	−2	−4	−2	−3	−3	−3	−3	−3	−1	−3	−3	0	0	0	−1	6	3	1
Y	−2	−2	−2	−3	−2	−3	−2	−3	−2	−1	2	−2	−2	−1	−1	−1	−1	3	7	2
W	−2	−3	−2	−4	−3	−2	−4	−4	−3	−2	−2	−3	−3	−1	−3	−2	−3	1	2	11

Figure 8.3: BLOSUM62, the most widely used substitution matrix. The amino acids are ordered by chemical similarity; the black squares mark the groups (C | small: S T P A G | acidic and amide: N D E Q | basic: H R K | hydrophobic: M I L V | aromatic: F Y W). Blue = positive (more often than by chance), red = negative.

8.5 PAM versus BLOSUM, and How to Choose

PAM	BLOSUM
made from *global* alignments of closely related proteins	made from *local* alignments of only highly conserved subsequences
long-range PAMs are *extrapolated* from 1PAM	every matrix is based on *observations*, no extrapolation
higher number = more distant	higher number = more closely related

Both have the same goal, and for every PAM there is a BLOSUM "partner" that leads to similar results. Different sources pair them differently; the slides list three versions. Regl recommends the following one, because it is based on matrices with similar *relative entropy*:

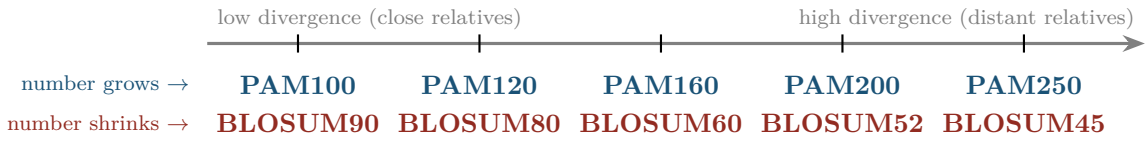


Figure 8.4: Corresponding PAM and BLOSUM matrices along the evolutionary distance axis (the pairing recommended in the lecture). PAM numbers increase with distance, BLOSUM numbers decrease.

Background: what "relative entropy" measures

The relative entropy $H = \sum_{a \le b} q_{ab}\, S(a,b)$ (with S in bits) is the average score per aligned pair in *truly related* sequences — how much information, on average, one aligned position carries to distinguish "related" from "random". A matrix for close relatives is "sharp" (high H: matches are expected, mismatches are suspicious); a matrix for distant relatives is "blunt" (low H). Matrices with the same H behave similarly, which is why they make good partners: BLOSUM62 and PAM160, for example, both carry about 0.7 bits per position.

8.5.1 A Rule of Thumb

First check the **sequence similarity** of the sequences you want to align, then pick the matrix (Table 8.1). For example, at about 30 % similarity, take BLOSUM60 or PAM160; the two give roughly the same answers.

Similarity (%)	PAM	BLOSUM
43	100	90
38	120	80
30	160	60
25	200	50
20	250	45

Table 8.1: Rule of thumb for selecting a matrix (it corresponds to the pairing in Fig. 8.4).

8.5.2 Which One? BLOSUM

Regl's advice — likely exam material

"Clearly BLOSUM!" Both have pros and cons, but PAM is around mainly for historical reasons ("maybe people use PAM because they were always using PAM") and has the drawbacks listed in Chapter 7. NCBI agrees: "A number of tests ... suggest that the BLOSUM matrices ... are superior to the PAM matrices for detecting biological relationships" (NCBI's BLAST tutorial).

In the tools. In BLAST (Section 9.2), the matrix is chosen under "Algorithm parameters → Scoring parameters". The choice is surprisingly small: PAM30, PAM70, PAM250, BLOSUM45, BLOSUM50, BLOSUM62 (the default), BLOSUM80 and BLOSUM90. Regl would drop the PAMs, offer more BLOSUMs and allow uploading your own matrix. ChimeraX (Section 9.4) offers many more PAMs and BLOSUMs: its 3D "matchmaker" starts with a *sequence* alignment, because regions with high sequence similarity are the ones it tries to superimpose first — and a sequence alignment needs a scoring matrix. The standard matrices can be downloaded from `ftp://ftp.ncbi.nih.gov/blast/matrices/`.

8.5.3 Standard Matrix or "Bring Your Own"?

All standard matrices were calculated from "general" proteins from across the databases. If you work on a special class of proteins, the honest answer is: calculate your own.

- **PAM**: possible in theory, but in practice nobody does it — dividing all sequences into families and finding the minimum-mutation trees is cumbersome, and PAM is disfavoured anyway.
- **BLOSUM**: possible in theory and in practice. Selecting the sequences is the main work; after that, computing the matrix is "a piece of cake" — programs are easily found or written (Section 8.3).

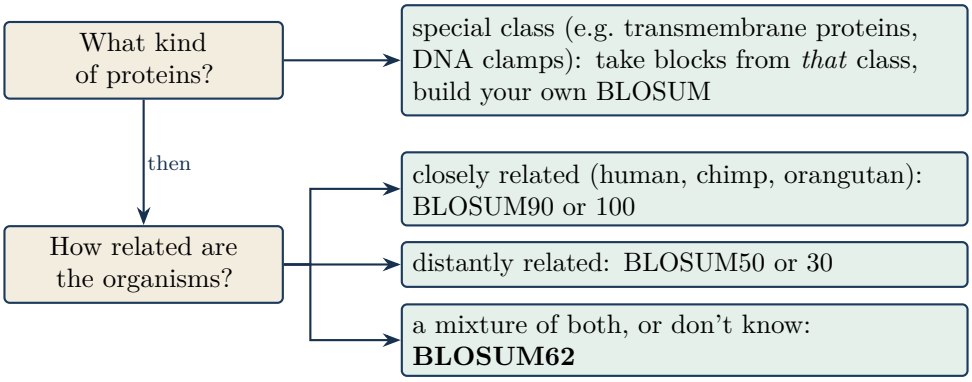


Figure 8.5: Choosing a scoring matrix: first orient it on the sequence type of your project, then on the sequence similarity.

BLOSUM62 is what people use when they are not really sure which one to take — it is "somehow in the middle" and was built from a collection of general proteins.

8.5.4 Alternatives

There are many more matrices, each mainly or only used for special purposes:

Matrix	Idea
MD	re-calculated PAM using up-to-date sequences
VTML	enhanced PAM with more sophisticated statistics behind the log-odds
PfSSM	special BLOSUM for the AT-rich genome of *Plasmodium falciparum* (the malaria parasite)
CBM, CCF	special BLOSUMs for transmembrane proteins
JTT, PHAT, SLIM, ...	for special classes of proteins, mostly derived from BLOSUM

All of them perform better in their field of application — but none of them is available in BLAST. Everybody uses BLOSUM (and some still use PAM).

8.6 Outlook: More Ideas on Scoring and Alignment

So far, every column of an alignment is scored with the *same* matrix, and alignments compare two sequences. The last slides of the scoring part point beyond that; the details follow in later chapters.

- **Profile-based scoring**: the alignment concentrates on functional sites across the whole sequence. This needs highly specialised algorithms and clever gap-scoring schemes.
- **Aligning groups of sequences**: like profile-based scoring, but first a profile (usually via majority vote, as in similarization) is constructed and used as a representative sequence (Chapter 12).
- **Position-specific scoring**: frequencies and log-odds are collected for *each position* of the alignment separately — the functional site gets its own, much stricter scores. This is used in some multiple sequence alignment software (and is the idea behind HMM profiles, Chapter 14).

One more type of alignment: overlaps. Sequencing machines cannot read a whole genome in one go; they produce millions of short *reads* (Chapter 3). In **genome assembly**, the reads are put together like a puzzle by finding pairs of reads whose ends overlap (Fig. 8.6). Neither a global alignment (the whole of both reads) nor a local one (any region) is right here: we need an alignment that *ignores gaps at the start of read i and at the end of read k*. The changes to the algorithms are quite straightforward (Chapter 10).

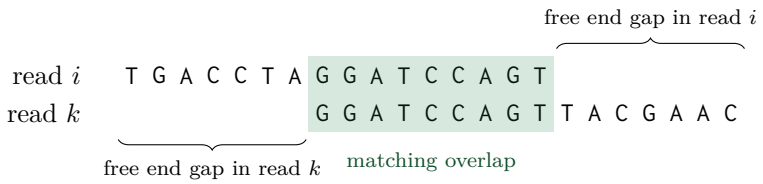


Figure 8.6: Overlap alignment of two sequencing reads: the end of read i matches the start of read k. The overhanging parts must not be penalised as gaps.

"This is not the end": there are also *repeated matches* and *nonlinear matches*, all implemented by small changes to the alignment algorithms.

8.7 Chapter Checklist

- why similarity comes in blocks (functional sites mutate slowly);
- what BLOSUM stands for, what a block is (same length, no indels) and what the x in BLOSUMx means — and that it runs opposite to PAM;
- elimination, similarization and weighting of close relatives;
- compute C (by counting and with the shortcut), T, Q, p, E, S and the rounded BLOSUM for a small block;
- read BLOSUM62: diagonal, conservative substitutions, examples;
- PAM vs BLOSUM: global/extrapolated vs local/observed; the correspondence and the rule of thumb; why BLOSUM is recommended;
- how to choose a matrix (sequence type, similarity, BLOSUM62 if unsure); when to build your own;
- profile-based and position-specific scoring; overlap alignment for genome assembly.

9

Tools in Practice: From Sequence to Structure

Lecture: VO04, "add-ons" deck (18 slides). Recording 00:00–31:00: a live demo of UniProt, BLAST, AlphaFold and ChimeraX with human and bovine insulin, followed by a real research case (METTL3 in bladder cancer).

The previous chapters explained *how* sequences are compared. This chapter shows the tools you actually click on. They come up again and again in the rest of the course, and Regl's question about BLAST is a classic in exams. The running example asks a simple question: *how similar are human and bovine insulin?* Insulin has the same function in humans and in cows, but over time mutations have appeared.

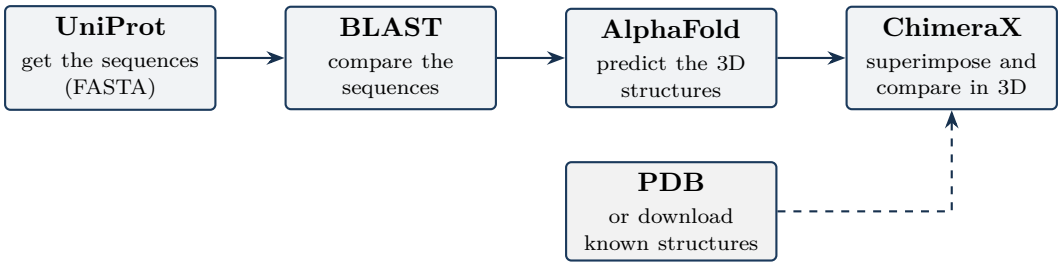


Figure 9.1: The workflow of the lecture demo — and, as we will see, of a real research project. Sequence first, then structure.

9.1 Databases: Where Sequences and Structures Live

Database	Contains	Notes
UniProt (`uniprot.org`)	**sequences**, mainly proteins, with everything known about them	run by EMBL-EBI together with the Swiss Institute of Bioinformatics and PIR
PDB (Protein Data Bank)	experimentally determined **3D structures**	entries have 4-character IDs such as `1AXC` (Chapter 4)
NCBI	the US National Center for Biotechnology Information: sequence databases (e.g. GenBank) and tools	home of **BLAST**
AlphaFold DB	about 200 million **predicted** structures	computed from UniProt sequences (Section 9.3)

Don't mix them up

UniProt contains *sequences*, the PDB contains *structures*. (And neither of them is run by NCBI.)

Getting the insulin sequences. Searching UniProt for "insulin" gives about 292,000 results — restricted to human, still nearly 12,000, because insulin appears in many configurations, bound to many different molecules (the same overrepresentation problem BLOSUM has to deal with, Chapter 8). The first hit is the one we want: human insulin, 110 amino acids. On its page you find everything that is known about it; we only need the sequence, so we download it in FASTA format (Chapter 4):

```
>sp|P01308|INS_HUMAN Insulin OS=Homo sapiens OX=9606 GN=INS PE=1 SV=1
MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAED
LQVGQVELGGGPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN
```

The first line is the identifier and some information (here: the UniProt accession `P01308`, the organism, the gene name), the following lines the sequence, 60 letters per line. The same for bovine insulin (105 amino acids).

9.2 BLAST

BLAST = Basic Local Alignment Search Tool

NCBI's tool for comparing a sequence (the **query**) against all sequences of a database, or against a second sequence (the **subject**). It returns the best **local** alignments (Chapter 11) together with statistics on how significant they are. Variants: `blastp` for proteins, `blastn` for nucleotides, and a few more.

Exam classic

Regl once asked a student in a bachelor exam whether BLAST produces local or global alignments, and she did not know. The answer is in the name: Basic **Local** Alignment Search Tool.

Normally BLAST checks your sequence against a whole database ("which known proteins look like mine?"). For two given sequences, tick "Align two or more sequences", paste or upload the two FASTA files and run protein BLAST. Under "Algorithm parameters" you can choose the scoring matrix (default BLOSUM62, Section 8.5).

9.2.1 Reading the Output

```
Query  1  MALWMRLLPLLALLALWGPDPAAAFVNQHLCGSHLVEALYLVCGERGFFYTPKTRREAED  60
          MALW RL PLLALLALW P PA AFVNQHLCGSHLVEALYLVCGERGFFYTPK RRE E
Sbjct  1  MALWTRLRPLLALLALWPPPPARAFVNQHLCGSHLVEALYLVCGERGFFYTPKARREVEG  60

Query 61  LQVGQVELGGGPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN           110
           QVG +EL     AG      LEG  QKRGIVEQCC S+CSLYQLENYCN
Sbjct 61  PQVGALEL-----AGGPGAGGLEGPPQKRGIVEQCCASVCSLYQLENYCN           105
```

- The first line is the query, the third line the subject; the numbers are positions in each sequence.
- The **middle line** shows the letter if both are identical, a + if they are different but *similar*

(positive score in the scoring matrix, e.g. V–L and I–V: +1 and +3 in BLOSUM62), and a **blank** if they are dissimilar.

- A **minus sign** in a sequence is a gap (insertion or deletion).

One look at the middle line tells you how similar the sequences are: lots of blanks, not similar. Here: 84 of 110 positions identical (76 %), 86 similar (78 %, "positives"), and one gap of 5 (BLAST reports exactly these three numbers above every alignment).

Where are the differences — and why there

The 26 differences are not spread evenly (Fig. 9.2). To understand the pattern you need one piece of biology: the 110 amino acids are *preproinsulin*, the precursor. The first 24 (the *signal peptide*) only direct the new protein into the cell's export route and are cut off right away. Then the middle part, the **C-peptide**, is cut out too. The actual hormone consists only of the **B chain** (30 amino acids) and the **A chain** (21), held together by disulfide bonds between their cysteines.

The differences sit where they do not matter: 18 in the C-peptide (which is thrown away) and 5 in the signal peptide, but only **3 in the 51 amino acids of the hormone itself**. Functional parts are conserved (Fig. 8.1) — and that is why insulin from cattle (and pigs) could be used to treat diabetes for about 60 years, until human insulin could be produced by genetically modified bacteria in the 1980s.

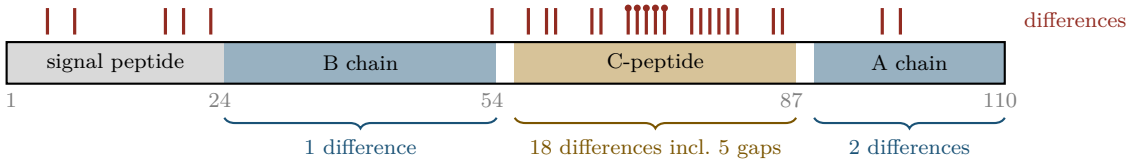


Figure 9.2: Human vs bovine preproinsulin (positions of the human sequence). Each red tick is a position where the BLAST alignment shows a difference (dots: the 5 gaps). The mature hormone (blue: B and A chain) is almost identical.

9.3 AlphaFold

If you search for AlphaFold, you get two different things:

- the **AlphaFold Protein Structure Database**: pre-computed structures for practically all sequences in UniProt and other sources — about 200 million. Look here first;
- the **AlphaFold Server** (`alphafoldserver.com`): paste your own sequence and get a prediction from scratch. Needed for sequences that are not in the database — for example a mutant (Section 9.5).

The predicted structure has the same file format as an experimental PDB entry, so all structure tools can use both.

9.3.1 How Much to Trust a Prediction

AlphaFold colours its prediction by a per-residue confidence score (pLDDT, from 0 to 100, Fig. 9.3).

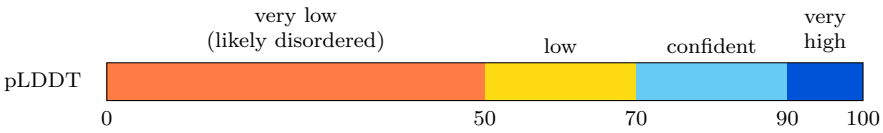


Figure 9.3: AlphaFold's confidence colour code: dark blue = high confidence, yellow and orange = very unsure.

The typical picture — for insulin as for almost any protein — is a confident (blue) **compact core** and uncertain (orange) **loops** sticking out. This makes physical sense: in a compact structure, every amino acid is held in place by many hydrogen bonds and van der Waals interactions with its neighbours, so its position is well determined. A loop on the surface could be in many positions ("this one, or this one, or this one"), and the predictor is honestly unsure. In the insulin prediction, even a small α-helix is coloured orange: AlphaFold does not really know whether it is a helix at all.

Each run produces **five predictions**, which are similar but slightly different. Comparing them is a good way to see how stable a prediction is.

Figure to insert: AlphaFold prediction of human insulin, coloured by confidence

Search the AlphaFold DB (`alphafold.ebi.ac.uk`) for `P01308` (human insulin) and take a screenshot of the 3D view: blue core, orange loops. Alternatively paste the sequence into alphafoldserver.com.

Figure 9.4: AlphaFold's prediction of human (pre-pro)insulin: confident core, uncertain loops.

9.4 ChimeraX

ChimeraX (`cgl.ucsf.edu/chimerax`), maintained by UCSF, is state-of-the-art molecular visualisation and analysis software: 3D matching of several proteins and "tons of chemical calculations". It is free — Regl's recommendation: "Just download it, you will use it over and over again." Don't confuse it with *Chimera*, its predecessor.

Superimposing two structures with matchmaker. Open both structures (from AlphaFold or the PDB). They are already roughly aligned, but not well. The *matchmaker* tool superimposes them. It asks which structure is the reference (with two it does not matter; with ten, all others are oriented on the one you pick). Internally, matchmaker first computes a *sequence* alignment — which is why it needs a scoring matrix (Section 8.5) — and then fits the 3D coordinates, starting with the regions where the sequences agree best.

For the two insulins, the best fit is achieved in the middle part of the prediction (a "curly" stretch that is neither a clear helix nor a sheet); the two α-helices at the ends end up in different positions, and the loops differ as well.

How close is close. Superpositions are judged by the average distance between matching residues, in **ångström** (1 Å $= 10^{-10}$ m; a carbon–carbon bond is about 1.5 Å long). Below 1 Å, two structures are practically identical at that position.

Figure to insert: ChimeraX matchmaker: human vs bovine insulin superimposed

To make it yourself in ChimeraX: `alphafold fetch P01308`, `alphafold fetch P01317` (bovine insulin), then `matchmaker #2 to #1`. Or search for "ChimeraX matchmaker superposition" and take an example image.

Figure 9.5: Two predicted insulin structures superimposed with ChimeraX's matchmaker.

Alternatives. **PyMOL** (`pymol.org`): powerful, originates from the UNIX world (keyboard shortcuts!), recently got a modern user interface. **TM-align** (`zhanggroup.org/TM-align`): 3D alignment only, also works with low sequence similarity, but only two proteins at a time.

9.5 A Real Case: METTL3 and Bladder Cancer

The insulin comparison is a toy example. The same workflow is used in a current research project in which Regl is involved: a group of medical doctors from the UK and Austria researching bladder cancer.

The starting point. The doctors found two families in which bladder cancer had been inherited over several generations (Fig. 9.6). Such a pedigree immediately suggests a genetic predisposition. Moreover, in all (or most) cancer cases, the same protein carried the same mutation, and the healthy family members did not.

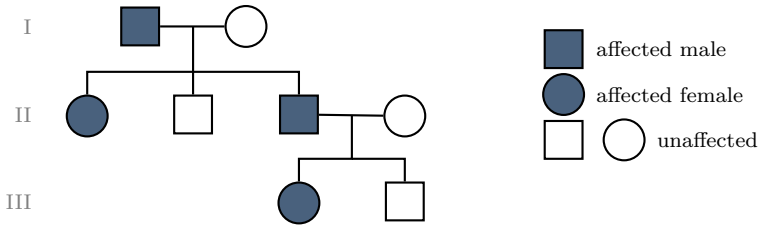


Figure 9.6: How to read a pedigree (a made-up schematic, not the real family): squares are men, circles women, filled symbols affected; a horizontal line between two people is a couple, the lines below lead to their children. A disease that appears generation after generation points to an inherited cause.

The proteins. METTL3 and METTL14 form a complex that attaches to RNA molecules and adds a methyl group ($-CH_3$) to one of its bases, adenine ("methylation"). Such chemical marks on RNA influence what happens to it in the cell — compare the marks on DNA in epigenetics (Section 2.3.5). The mutation is in METTL3: **one arginine (R) is missing** compared with the *wild type* (the normal, non-mutated sequence):

```
...YGTLTDDEMRRLNIP...  (wild type)
...YGTLTDDEM-RLNIP...  (mutant)
```

(Since there are two R's in a row, the gap could equally be placed under the other one; both alignments have the same score.) The question: *does this deletion influence the binding of the complex to RNA?* The doctors had no experience with bioinformatics tools, so they asked Regl.

The approach. Exactly the insulin workflow (Fig. 9.1), first for METTL3 only: get the sequences of wild type and mutant (UniProtKB); test whether the PDB already has structures; check with BLAST that the sequences really differ in only this one position; infer structures for both with AlphaFold; check alternatives (Rosetta and trRosetta, Boltz, ESMFold, Modeller); compare them with ChimeraX.

What came out.

- There is no pre-computed structure for the mutant, and AlphaFold's results are "not really convincing": within the three domains the prediction is stable (dark blue), outside them it is not (yellow, orange).
- A prediction from trRosetta (a variant of Rosetta, a predictor that has been around for many years) looks quite different.
- AlphaFold's five predictions disagree strongly in the connecting parts between the domains: regions that should be aligned end up in very different places.
- *Within* the domains, however, the results are good: the average distance between matching residues of the five predictions is mostly well below 1 Å (Table 9.1).

	1	2	3	4
0	0.237	0.372	1.395	0.211
1	–	0.453	1.178	0.129
2	–	–	0.366	0.388
3	–	–	–	0.119

Table 9.1: Average distance (in Å) between matching residues of AlphaFold's five predictions (0–4) for the wild type, in the regions with the best similarity (not all residues).

So the connecting sequences were cut out, and the analysis concentrated on the **large domain**: it gives the best 3D prediction, and it contains the functional site — and the mutation. Aligning wild type and mutant first showed a striking dissimilarity, but it seems to be caused by the uncertainties in the connecting parts. The main difference found so far: **the mutation creates a short α-helix** at the position of the missing arginine. This seems to be a key issue.

Open questions and outlook. What happens in the METTL3/METTL14 complex? Does the change influence the binding to RNA? The next step is to model the complex together with RNA and test how close the contacts are. If a conformational reason is found why the mutation causes cancer, one could think of *gene editing* (re-inserting the missing arginine) or of medication that compensates the negative effects — a direct implication for public health.

The lesson

Real research uses the same handful of tools as the toy example. But predicted structures are hypotheses, not facts: always check the confidence colours, compare several predictions and predictors, and restrict your conclusions to the regions where they agree.

9.6 Chapter Checklist

- UniProt (sequences) vs PDB (structures) vs NCBI (BLAST); AlphaFold DB vs AlphaFold Server;
- what BLAST stands for and that it produces *local* alignments; query and subject; read the middle line (letter, +, blank, −);

- why human and bovine insulin differ mostly outside the hormone itself;
- AlphaFold's confidence colours; why cores are predicted well and loops badly; five predictions per run;
- ChimeraX and matchmaker (sequence alignment first, then 3D fit); alternatives PyMOL and TM-align;
- the METTL3 case: question, workflow, the main finding and its limits.

10

Pairwise Alignment I: Global Alignment

Lecture: Lecture on 16.11. — not yet held. This chapter is an outline.

Chapter 5 showed that there are far too many alignments to try them all ($\approx 10^{58}$ for two sequences of length 100). Global alignment finds the best one anyway, with *dynamic programming*: the best alignment of two sequences is built from the best alignments of their prefixes, stored in a table and filled in row by row — the same idea as the edit distance you may know from algorithms courses.

Planned content: the Needleman–Wunsch algorithm (score matrix, recursion with match, gap in x, gap in y; traceback); a small example filled in by hand; linear vs affine gap costs; time and memory $O(nm)$; end gaps and the overlap alignment from Section 8.6.

11

Pairwise Alignment II: Local Alignment

Lecture: Lecture on 23.11. — not yet held. This chapter is an outline.

Often only a part of two sequences is related — a shared domain or a functional site (Fig. 8.1). Local alignment finds the best-scoring pair of *subsequences*; it is what BLAST does (Section 9.2).

Planned content: the Smith–Waterman algorithm (the 0 in the recursion, traceback from the maximum); why the scoring matrix must have a negative expected score; heuristics: the idea behind BLAST (seeds, extension) and FASTA; significance of hits (E-values).

12

Multiple Sequence Alignment

Lecture: Lectures on 30.11. and 7.12. — not yet held. This chapter is an outline.

Aligning many sequences at once is what the blocks of BLOSUM (Chapter 8), the family trees of PAM (Chapter 7) and AlphaFold's input (Section 3.13) are built on.

Planned content: why exact dynamic programming does not scale to many sequences; sum-of-pairs scores; progressive alignment along a guide tree (Clustal); profiles and position-specific scoring (Section 8.6); typical tools.

13

Phylogenetics

Lecture: Lecture on 14.12. — not yet held. This chapter is an outline.

Similarity of sequences reflects relationship (Chapter 4); phylogenetics turns pairwise similarities into family trees, like the hemoglobin tree in Chapter 4 or Dayhoff's trees for PAM (Fig. 7.2).

Planned content: distances from alignments; distance-based tree building (UPGMA, neighbour joining); parsimony; rooted vs unrooted trees; the molecular clock revisited.

14

Hidden Markov Models

Lecture: Lecture on 21.12. — not yet held. This chapter is an outline.

Markov chains already appeared with PAM (Section 7.6). Hidden Markov models add states that are not observed directly — for example "inside a gene" vs "between genes", or the positions of a protein family profile.

Planned content: states, transition and emission probabilities; the Viterbi algorithm (again dynamic programming); profile HMMs for protein families.

15

AI-Based Biological Sequence Models

Lecture: Lectures on 11.01. and 18.01. — not yet held. This chapter is an outline.

The course ends where it began (Chapter 1): with machine learning on the "languages of life".

Planned content: protein language models; how AlphaFold uses multiple sequence alignments; structure prediction and its confidence (Section 9.3); applications.